SDK REFERENCES

Agent SDK reference - TypeScript

Complete API reference for the TypeScript Agent SDK, including all functions, types, and interfaces.

Installation

```
npm install @anthropic-ai/claude-agent-sdk
```

The SDK bundles a native Claude Code binary for your platform as an optional dependency such as `@anthropic-ai/claude-agent-sdk-darwin-arm64`. You don't need to install Claude Code separately. If your package manager skips optional dependencies, the SDK throws `Native CLI binary for <platform> not found`; set `pathToClaudeCodeExecutable` to a separately installed `claude` binary instead.

Compile to a single executable

When you compile your application into a single-file executable with `bun build --compile`, the SDK cannot resolve the bundled CLI binary at runtime. `require.resolve` does not work inside the compiled executable's `$bunfs` virtual filesystem, so the SDK throws `Native CLI binary for <platform> not found`.

To work around this, embed the platform binary as a file asset, extract it to a real path at startup with `extractFromBunfs()`, and pass that path to `pathToClaudeCodeExecutable`.

The `extractFromBunfs()` helper requires `@anthropic-ai/claude-agent-sdk` v0.3.144 or later. The example below builds for macOS on Apple Silicon:

```
import binPath from "@anthropic-ai/claude-agent-sdk-darwin-
arm64/claude" with { type: "file" };
import { extractFromBunfs } from "@anthropic-ai/claude-agent-
sdk/extract";
import { query } from "@anthropic-ai/claude-agent-sdk";

const cliPath = extractFromBunfs(binPath);

for await (const message of query({
  prompt: "Hello",
  options: { pathToClaudeCodeExecutable: cliPath },
})) {
  console.log(message);
}
```

`extractFromBunfs()` copies the embedded binary out of the compiled executable's virtual filesystem to a per-user temp directory and returns the real path. Outside a compiled executable it returns the input path unchanged, so the same code runs in development without modification.

Each compiled executable embeds a single platform's binary. Match the platform package in the import to your `--target`:

- To cross-compile, install the non-matching platform package, for example `npm install @anthropic-ai/claude-agent-sdk-linux-x64 --force`.
- On Windows, the binary subpath is `claude.exe`, for example `@anthropic-ai/claude-agent-sdk-win32-x64/claude.exe`.

Functions

`query()`

The primary function for interacting with Claude Code. Creates an async generator that streams messages as they arrive.

```
function query({
  prompt,
  options
}: {
  prompt: string | AsyncIterable<SDKUserMessage>;
  options?: Options;
}): Query;
```

Parameters

Parameter	Type	Description
`prompt`	`string | AsyncIterable<SDKUserMessage>`	The input prompt as a string or async iterable for streaming mode
`options`	`Options`	Optional configuration object (see Options type below)

Returns

Returns a `Query` object that extends `AsyncGenerator<SDKMessage, void>` with additional methods.

`startup()`

Pre-warms the CLI subprocess by spawning it and completing the initialize handshake before a prompt is available. The returned `WarmQuery` handle accepts a prompt later and writes it to an already-ready process, so the first `query()` call resolves without paying subprocess spawn and initialization cost inline.

```
function startup(params?: {
  options?: Options;
  initializeTimeoutMs?: number;
}): Promise<WarmQuery>;
```

Parameters

Parameter	Type	Description
`options`	`Options`	Optional configuration object. Same as the `options` parameter to `query()`
`initializeTimeoutMs`	`number`	Maximum time in milliseconds to wait for subprocess initialization. Defaults to `60000`. If initialization does not complete in time, the promise rejects with a timeout error

Returns

Returns a `Promise<WarmQuery>` that resolves once the subprocess has spawned and completed its initialize handshake.

Example

Call `startup()` early, for example on application boot, then call `.query()` on the returned handle once a prompt is ready. This moves subprocess spawn and initialization out of the critical path.

```
import { startup } from "@anthropic-ai/claude-agent-sdk";

// Pay startup cost upfront
const warm = await startup({ options: { maxTurns: 3 } });

// Later, when a prompt is ready, this is immediate
for await (const message of warm.query("What files are here?")) {
  console.log(message);
}
```

`tool()`

Creates a type-safe MCP tool definition for use with SDK MCP servers.

```
function tool<Schema extends AnyZodRawShape>(
  name: string,
  description: string,
  inputSchema: Schema,
  handler: (args: InferShape<Schema>, extra: unknown) =>
Promise<CallToolResult>,
  extras?: { annotations?: ToolAnnotations }
): SdkMcpToolDefinition<Schema>;
```

Parameters

Parameter	Type	Description
`name`	`string`	The name of the tool
`description`	`string`	A description of what the tool does
`inputSchema`	`Schema extends AnyZodRawShape`	Zod schema defining the tool's input parameters (supports both Zod 3 and Zod 4)
`handler`	`(args, extra) => Promise<CallToolResult>`	Async function that executes the tool logic
`extras`	`{ annotations?: ToolAnnotations }`	Optional MCP tool annotations providing behavioral hints to clients

`ToolAnnotations`

Re-exported from `@modelcontextprotocol/sdk/types.js`. All fields are optional hints; clients should not rely on them for security decisions.

Field	Type	Default	Description
`title`	`string`	`undefined`	Human-readable title for the tool
`readOnlyHint`	`boolean`	`false`	If `true`, the tool does not modify its environment
`destructiveHint`	`boolean`	`true`	If `true`, the tool may perform destructive updates (only meaningful when `readOnlyHint` is `false`)
`idempotentHint`	`boolean`	`false`	If `true`, repeated calls with the same arguments have no additional effect (only meaningful when `readOnlyHint` is `false`)
`openWorldHint`	`boolean`	`true`	If `true`, the tool interacts with external

Field	Type	Default	Description
			entities (for example, web search). If `false`, the tool's domain is closed (for example, a memory tool)

```
import { tool } from "@anthropic-ai/claude-agent-sdk";
import { z } from "zod";

const searchTool = tool(
  "search",
  "Search the web",
  { query: z.string() },
  async ({ query }) => {
    return { content: [{ type: "text", text: `Results for:
${query}` }] };
  },
  { annotations: { readOnlyHint: true, openWorldHint: true } }
);
```

`createSdkMcpServer()`

Creates an MCP server instance that runs in the same process as your application.

```
function createSdkMcpServer(options: {
  name: string;
  version?: string;
  tools?: Array<SdkMcpToolDefinition<any>>;
}): McpSdkServerConfigWithInstance;
```

Parameters

Parameter	Type	Description
`options.name`	`string`	The name of the MCP server
`options.version`	`string`	Optional version string
`options.tools`	`Array<SdkMcpToolDefinition>`	Array of tool definitions created with `tool()`

`listSessions()`

Discovers and lists past sessions with light metadata. Filter by project directory or list sessions across all projects.

```
function listSessions(options?: ListSessionsOptions):
Promise<SDKSessionInfo[]>;
```

Parameters

Parameter	Type	Default	Description
`options.dir`	`string`	`undefined`	Directory to list sessions for. When omitted, returns sessions across all projects
`options.limit`	`number`	`undefined`	Maximum number of sessions to return
`options.includeWorktrees`	`boolean`	`true`	When `dir` is inside a git repository, include sessions from all worktree paths

Return type: `SDKSessionInfo`

Property	Type	Description
`sessionId`	`string`	Unique session identifier (UUID)
`summary`	`string`	Display title: custom title, auto-generated summary, or first prompt
`lastModified`	`number`	Last modified time in milliseconds since epoch
`fileSize`	`number | undefined`	Session file size in bytes. Only populated for local JSONL storage
`customTitle`	`string | undefined`	User-set session title (via `/rename`)
`firstPrompt`	`string | undefined`	First meaningful user prompt in the session
`gitBranch`	`string | undefined`	Git branch at the end of the session
`cwd`	`string | undefined`	Working directory for the session
`tag`	`string |`	User-set session tag (see `tagSession()`)

Property	Type	Description
	`undefined`	
`createdAt`	`number | undefined`	Creation time in milliseconds since epoch, from the first entry's timestamp

Example

Print the 10 most recent sessions for a project. Results are sorted by `lastModified` descending, so the first item is the newest. Omit `dir` to search across all projects.

```
import { listSessions } from "@anthropic-ai/claude-agent-sdk";

const sessions = await listSessions({ dir: "/path/to/project",
limit: 10 });

for (const session of sessions) {
  console.log(`${session.summary} (${session.sessionId})`);
}
```

`getSessionMessages()`

Reads user and assistant messages from a past session transcript.

```
function getSessionMessages(
  sessionId: string,
  options?: GetSessionMessagesOptions
): Promise<SessionMessage[]>;
```

Parameters

Parameter	Type	Default	Description
`sessionId`	`string`	required	Session UUID to read (see `listSessions()`)
`options.dir`	`string`	`undefined`	Project directory to find the session in. When omitted, searches all projects
`options.limit`	`number`	`undefined`	Maximum number of messages to return
`options.offset`	`number`	`undefined`	Number of messages to skip from the start

Return type: `SessionMessage`

Property	Type	Description
`type`	`"user" | "assistant"`	Message role
`uuid`	`string`	Unique message identifier
`session_id`	`string`	Session this message belongs to
`message`	`unknown`	Raw message payload from the transcript
`parent_tool_use_id`	`string | null`	For subagent messages, the `tool_use_id` of the spawning `Agent` tool call. `null` for main-session messages and older sessions

Example

```
import { listSessions, getSessionMessages } from "@anthropic-
ai/claude-agent-sdk";

const [latest] = await listSessions({ dir: "/path/to/project",
limit: 1 });

if (latest) {
  const messages = await getSessionMessages(latest.sessionId, {
    dir: "/path/to/project",
    limit: 20
  });

  for (const msg of messages) {
    console.log(`[${msg.type}] ${msg.uuid}`);
  }
}
```

`getSessionInfo()`

Reads metadata for a single session by ID without scanning the full project directory.

```ts
function getSessionInfo(
  sessionId: string,
  options?: GetSessionInfoOptions
): Promise<SDKSessionInfo | undefined>;
```

Parameters

Parameter	Type	Default	Description
`sessionId`	`string`	required	UUID of the session to look up
`options.dir`	`string`	`undefined`	Project directory path. When omitted, searches all project directories

Returns `SDKSessionInfo`, or `undefined` if the session is not found.

`renameSession()`

Renames a session by appending a custom-title entry. Repeated calls are safe; the most recent title wins.

```ts
function renameSession(
  sessionId: string,
  title: string,
  options?: SessionMutationOptions
): Promise<void>;
```

Parameters

Parameter	Type	Default	Description
`sessionId`	`string`	required	UUID of the session to rename
`title`	`string`	required	New title. Must be non-empty after trimming whitespace
`options.dir`	`string`	`undefined`	Project directory path. When omitted, searches all project directories

`tagSession()`

Tags a session. Pass `null` to clear the tag. Repeated calls are safe; the most recent tag wins.

```
function tagSession(
  sessionId: string,
  tag: string | null,
  options?: SessionMutationOptions
): Promise<void>;
```

Parameters

Parameter	Type	Default	Description
`sessionId`	`string`	required	UUID of the session to tag
`tag`	`string | null`	required	Tag string, or `null` to clear
`options.dir`	`string`	`undefined`	Project directory path. When omitted, searches all project directories

`resolveSettings()`

Resolves the effective Claude Code settings for a given directory using the same merge engine as the CLI, without spawning the Claude CLI. Use it to inspect what configuration a `query()` call would see before invoking one.

> This function is alpha and its API may change before stabilization. It reads MDM sources, including macOS plist and Windows HKLM/HKCU, for parity with CLI startup, but does not execute the admin-configured `policyHelper` subprocess. The `permissions.defaultMode` field is returned as-is from all tiers including project settings. The trust filter the CLI applies before honoring escalating permission modes is not applied.

```
function resolveSettings(
  options?: ResolveSettingsOptions
): Promise<ResolvedSettings>;
```

Parameters

`resolveSettings()` accepts a single options object. All fields are optional.

Parameter	Type	Default	Description
`options.cwd`	`string`	`process.cwd()`	Directory to resolve project and local settings relative to
`options.settingSources`	`SettingSource[]`	All sources	Which filesystem sources to load. Pass `[]` to skip user, project, and local settings. **Endpoint-managed policy** loads in all cases. Server-managed settings are taken from `serverManagedSettings` when the host passes it, or read from the CLI's on-disk cache otherwise; the snapshot does not fetch them from the network
`options.managedSettings`	`Settings`	`undefined`	Restrictive policy-tier settings supplied by the embedding host. Dropped by default when an admin-deployed managed tier is present; merged under that tier when `parentSettingsBehavior` is `"merge"`. Non-restrictive keys such as `model` are silently dropped so this option can tighten managed policy but not loosen it
`options.serverManagedSettings`	`Settings`	`undefined`	Server-managed settings payload from `/api/claude_code/settings`. Non-restrictive keys pass through unfiltered

Return type: `ResolvedSettings`

`resolveSettings()` returns an object describing the merged settings and the source that contributed each key.

Property	Type	Description
`effective`	`Settings`	Merged settings after applying all enabled sources in precedence order
`provenance`	`Partial<Record<keyof Settings, ProvenanceEntry>>`	For each top-level key in `effective`, which source supplied the value
`sources`	`Array<{ source, settings, path?, policyOrigin? }>`	Per-source raw settings, ordered from lowest to highest precedence

Example

The example below resolves settings for a project directory and prints the source that controls the cleanup period.

```typescript
import { resolveSettings } from "@anthropic-ai/claude-agent-sdk";

const { effective, provenance } = await resolveSettings({
  cwd: "/path/to/project",
  settingSources: ["user", "project", "local"],
});

console.log(`Cleanup period: ${effective.cleanupPeriodDays}
days`);
console.log(`Set by: ${provenance.cleanupPeriodDays?.source}`);
```

Types

`Options`

Configuration object for the `query()` function.

Property	Type	Default	Description
`abortController`	`AbortController`	`new AbortController()`	Controller for cancelling operations
`additionalDirectories`	`string[]`	`[]`	Additional directories Claude can access
`agent`	`string`	`undefined`	Agent name for the main thread. The agent must be defined in the `agents` option or in settings
`agents`	`Record<string, [ AgentDefinition ] (#agentdefinition)>`	`undefined`	Programmatically define subagents
`agentProgressSummaries`	`boolean`	`false`	When `true`, generate one-line progress summaries for subagents and forward them on `task_progress` events via the `summary` field. Applies to foreground and background subagents
`allowDangerouslySkipPermissions`	`boolean`	`false`	Enable bypassing permissions. Required when using `permissionMode: 'bypassPermissions'`
`allowedTools`	`string[]`	`[]`	Tools to auto-approve without prompting. This does not restrict Claude to only these tools; unlisted

Property	Type	Default	Description
			tools fall through to `permissionMode` and `canUseTool`. Use `disallowedTools` to block tools. See **Permissions**
`betas`	`SdkBeta[]`	`[]`	Enable beta features
`canUseTool`	`CanUseTool`	`undefined`	Custom permission function for tool usage
`continue`	`boolean`	`false`	Continue the most recent conversation
`cwd`	`string`	`process.cwd()`	Current working directory
`debug`	`boolean`	`false`	Enable debug mode for the Claude Code process
`debugFile`	`string`	`undefined`	Write debug logs to a specific file path. Implicitly enables debug mode
`disallowedTools`	`string[]`	`[]`	Tools to deny. A bare name such as `"Bash"` removes the tool from Claude's context. A scoped rule such as `"Bash(rm *)"` leaves the tool available and denies matching calls in every permission mode, including `bypassPermissions`. See **Permissions**
`effort`	`'low' | 'medium' | 'high' | 'xhigh' | 'max'`	Model default	Controls how much effort Claude puts into its response. Works with adaptive thinking to guide thinking depth. See **adjust the effort level**
`enableFileCheckpointing`	`boolean`	`false`	Enable file change tracking for rewinding. See **File checkpointing**
`env`	`Record<string, string | undefined>`	`process.env`	Environment variables. When set, this replaces the subprocess environment instead of merging with `process.env`, so pass `{ ...process.env, YOUR_VAR: 'value' }` to keep inherited variables like `PATH`. See **Handle slow or stalled API responses** for an example of this pattern, and **Environment variables** for variables the underlying CLI reads. Set `CLAUDE_AGENT_SDK_CLIENT_APP` to identify your app in the User-Agent header
`executable`	`'bun' | 'deno' | 'node'`	Auto-detected	JavaScript runtime to use

Property	Type	Default	Description
`executableArgs`	`string[]`	`[]`	Arguments to pass to the executable
`extraArgs`	`Record<string, string | null>`	`{}`	Additional arguments
`fallbackModel`	`string`	`undefined`	Model to use if primary fails
`forkSession`	`boolean`	`false`	When resuming with `resume`, fork to a new session ID instead of continuing the original session
`forwardSubagentText`	`boolean`	`false`	Forward subagent text and thinking blocks as assistant and user messages with `parent_tool_use_id` set, so consumers can render a nested transcript. By default only `tool_use` and `tool_result` blocks from subagents are emitted
`hooks`	`Partial<Record<HookEvent, HookCallbackMatcher[]>>`	`{}`	Hook callbacks for events
`includeHookEvents`	`boolean`	`false`	Include hook lifecycle events in the message stream as `SDKHookStartedMessage`, `SDKHookProgressMessage`, and `SDKHookResponseMessage`
`includePartialMessages`	`boolean`	`false`	Include partial message events
`loadTimeoutMs`	`number`	`60000`	*Alpha.* Timeout in milliseconds for each `sessionStore.load()` and `sessionStore.listSubkeys()` call during resume materialization. If the adapter doesn't settle within this window, the query fails instead of hanging. Ignored when `sessionStore` is not set
`managedSettings`	`Settings`	`undefined`	Policy-tier settings supplied by the spawning parent process. Dropped when an IT-controlled managed-settings tier already exists on the machine, unless that admin opts in with `parentSettingsBehavior: 'merge'`. Filtered to restrictive-only keys regardless
`maxBudgetUsd`	`number`	`undefined`	Stop the query when the client-side cost estimate reaches this USD value. Compared against the same estimate

Property	Type	Default	Description
			as `total_cost_usd`; see **Track cost and usage** for accuracy caveats
`maxThinkingTokens`	`number`	`undefined`	*Deprecated:* Use `thinking` instead. Maximum tokens for thinking process
`maxTurns`	`number`	`undefined`	Maximum agentic turns (tool-use round trips)
`mcpServers`	`Record<string, [ McpServerConfig ] (#mcpserverconfig)>`	`{}`	MCP server configurations
`model`	`string`	Default from CLI	Claude model alias or full model name. See **accepted values and provider-specific IDs**
`onElicitation`	`(request: ElicitationRequest, options: { signal: AbortSignal }) => Promise<ElicitationResult>`	`undefined`	Callback for handling MCP elicitation requests. Called when an MCP server requests user input and no hook handles it first. When not provided, unhandled elicitation requests are declined automatically
`outputFormat`	`{ type: 'json_schema', schema: JSONSchema }`	`undefined`	Define output format for agent results. See **Structured outputs** for details
`outputStyle`	`string`	`undefined`	Not an `Options` field. Set `outputStyle` in the inline `settings` object or a settings file instead. See **Activate an output style**
`pathToClaudeCodeExecutable`	`string`	Auto-resolved from bundled native binary	Path to Claude Code executable. Only needed if optional dependencies were skipped during install or your platform isn't in the supported set
`permissionMode`	`PermissionMode`	`'default'`	Permission mode for the session
`permissionPromptToolName`	`string`	`undefined`	MCP tool name for permission prompts
`persistSession`	`boolean`	`true`	When `false`, disables session persistence to disk. Sessions cannot be resumed later
`planModeInstructions`	`string`	`undefined`	Custom workflow instructions for plan mode. When `permissionMode` is `'plan'`, this string replaces the default plan-mode workflow body. The CLI still wraps it with the read-only enforcement preamble and the ExitPlanMode protocol footer

Property	Type	Default	Description
`plugins`	`SdkPluginConfig[]`	`[]`	Load custom plugins from local paths. See **Plugins** for details
`promptSuggestions`	`boolean`	`false`	Enable prompt suggestions. Emits a `prompt_suggestion` message after each turn with a predicted next user prompt
`resume`	`string`	`undefined`	Session ID to resume
`resumeSessionAt`	`string`	`undefined`	Resume session at a specific message UUID
`sandbox`	`SandboxSettings`	`undefined`	Configure sandbox behavior programmatically. See **Sandbox settings** for details
`sessionId`	`string`	Auto-generated	Use a specific UUID for the session instead of auto-generating one
`sessionStore`	`SessionStore`	`undefined`	Mirror session transcripts to an external backend so any host can resume them. See **Persist sessions to external storage**
`sessionStoreFlush`	`'batched' | 'eager'`	`'batched'`	*Alpha.* Flush mode for `sessionStore`. Ignored when `sessionStore` is not set
`settings`	`string | Settings`	`undefined`	Inline **settings** object or path to a settings file. Populates the flag-settings layer in the **precedence order**. Change at runtime with `applyFlagSettings()`
`settingSources`	`SettingSource[]`	CLI defaults (all sources)	Control which filesystem settings to load. Pass `[]` to disable user, project, and local settings. **Endpoint-managed policy** loads regardless; server-managed settings are fetched when the session authenticates with an organization credential on an **eligible configuration**. See **Use Claude Code features**
`skills`	`string[] | 'all'`	`undefined`	Skills available to the session. Pass `'all'` to enable every discovered skill, or a list of skill names. When set, the SDK adds the Skill tool to `allowedTools` automatically. If you also pass `tools`, include `'Skill'` in that list. See **Skills**

Property	Type	Default	Description
`spawnClaudeCodeProcess`	`(options: SpawnOptions) => SpawnedProcess`	`undefined`	Custom function to spawn the Claude Code process. Use to run Claude Code in VMs, containers, or remote environments
`stderr`	`(data: string) => void`	`undefined`	Callback for stderr output
`strictMcpConfig`	`boolean`	`false`	Use only the servers passed in `mcpServers` and ignore project `.mcp.json`, user settings, plugin-provided MCP servers, and **claude.ai connectors**
`systemPrompt`	`string | { type: 'preset'; preset: 'claude_code'; append?: string; excludeDynamicSections?: boolean }`	`undefined` (minimal prompt)	System prompt configuration. Pass a string for custom prompt, or `{ type: 'preset', preset: 'claude_code' }` to use Claude Code's system prompt. When using the preset object form, add `append` to extend it with additional instructions, and set `excludeDynamicSections: true` to move per-session context into the first user message for **better prompt-cache reuse across machines**
`taskBudget`	`{ total: number }`	`undefined`	*Alpha.* API-side task budget in tokens. When set, the model is told its remaining token budget so it can pace tool use and wrap up before the limit
`thinking`	`ThinkingConfig`	`{ type: 'adaptive' }` for supported models	Controls Claude's thinking/reasoning behavior. See `ThinkingConfig` for options
`title`	`string`	`undefined`	Display title for the session. When resuming via `resume` or `continue`, the resumed session's persisted title takes precedence; use `renameSession()` to retitle an existing session
`toolAliases`	`Record<string, string>`	`undefined`	Map built-in tool names to MCP tool names so Claude calls your MCP implementation in place of the built-in. For example, `{ Bash: 'mcp__workspace__bash' }`
`toolConfig`	`ToolConfig`	`undefined`	Configuration for built-in tool behavior. See `ToolConfig` for details
`tools`	`string[] | { type: 'preset'; preset:`	`undefined`	Tool configuration. Pass an array of tool names or use the preset to get

Property	Type	Default	Description
	`'claude_code' }`		Claude Code's default tools

Handle slow or stalled API responses

The CLI subprocess reads several environment variables that control API timeouts and stall detection. Pass them through the `env` option:

```
const result = query({
  prompt: "Analyze this code",
  options: {
    env: {
      ...process.env,
      API_TIMEOUT_MS: "120000",
      CLAUDE_CODE_MAX_RETRIES: "2",
      CLAUDE_ASYNC_AGENT_STALL_TIMEOUT_MS: "120000",
    },
  },
});
```

- `API_TIMEOUT_MS`: per-request timeout on the Anthropic client, in milliseconds. Default `600000`. Applies to the main loop and all subagents.
- `CLAUDE_CODE_MAX_RETRIES`: maximum API retries. Default `10`, capped at `15`. Each retry gets its own `API_TIMEOUT_MS` window, so worst-case wall time is roughly `API_TIMEOUT_MS × (CLAUDE_CODE_MAX_RETRIES + 1)` plus backoff. For unattended runs that need to wait through longer outages, set `CLAUDE_CODE_RETRY_WATCHDOG=1` to retry capacity errors indefinitely.
- `CLAUDE_ASYNC_AGENT_STALL_TIMEOUT_MS`: stall watchdog for subagents launched with `run_in_background`. Default `600000`. Resets on each stream event; on stall it aborts the subagent, marks the task failed, and surfaces the error to the parent with any partial result. Does not apply to synchronous subagents.
- `CLAUDE_ENABLE_STREAM_WATCHDOG=1` with `CLAUDE_STREAM_IDLE_TIMEOUT_MS`: aborts the request when headers have arrived but the response body stops streaming. When `CLAUDE_ENABLE_STREAM_WATCHDOG` is unset, the default is server-controlled on the direct Anthropic API and off on other providers. `CLAUDE_STREAM_IDLE_TIMEOUT_MS` defaults to `300000` and is clamped to that minimum. The aborted request goes through the normal retry path.

`Query` object

Interface returned by the `query()` function.

```
interface Query extends AsyncGenerator<SDKMessage, void> {
  interrupt(): Promise<void>;
  rewindFiles(
    userMessageId: string,
    options?: { dryRun?: boolean }
  ): Promise<RewindFilesResult>;
  setPermissionMode(mode: PermissionMode): Promise<void>;
  setModel(model?: string): Promise<void>;
  setMaxThinkingTokens(maxThinkingTokens: number | null):
Promise<void>;
  applyFlagSettings(settings: { [K in keyof Settings]?:
Settings[K] | null }): Promise<void>;
  initializationResult(): Promise<SDKControlInitializeResponse>;
  supportedCommands(): Promise<SlashCommand[]>;
  supportedModels(): Promise<ModelInfo[]>;
  supportedAgents(): Promise<AgentInfo[]>;
  mcpServerStatus(): Promise<McpServerStatus[]>;
  accountInfo(): Promise<AccountInfo>;
  reconnectMcpServer(serverName: string): Promise<void>;
  toggleMcpServer(serverName: string, enabled: boolean):
Promise<void>;
  setMcpServers(servers: Record<string, McpServerConfig>):
Promise<McpSetServersResult>;
  streamInput(stream: AsyncIterable<SDKUserMessage>):
Promise<void>;
  stopTask(taskId: string): Promise<void>;
  close(): void;
}
```

Methods

Method	Description
`interrupt()`	Interrupts the query (only available in streaming input mode)
`rewindFiles(userMessageId, options?)`	Restores files to their state at the specified user message. Pass `{ dryRun: true }` to preview changes. Requires `enableFileCheckpointing: true`. See **File checkpointing**
`setPermissionMode()`	Changes the permission mode (only available in streaming input mode)

Method	Description
`setModel()`	Changes the model (only available in streaming input mode)
`setMaxThinkingTokens()`	*Deprecated:* Use the `thinking` option instead. Changes the maximum thinking tokens
`applyFlagSettings(settings)`	Merges settings into the session's flag settings layer at runtime (only available in streaming input mode). See `applyFlagSettings()`
`initializationResult()`	Returns the full initialization result including supported commands, models, account info, and output style configuration
`supportedCommands()`	Returns available slash commands
`supportedModels()`	Returns available models with display info
`supportedAgents()`	Returns available subagents as `AgentInfo[]`
`mcpServerStatus()`	Returns status of connected MCP servers
`accountInfo()`	Returns account information
`reconnectMcpServer(serverName)`	Reconnect an MCP server by name
`toggleMcpServer(serverName, enabled)`	Enable or disable an MCP server by name
`setMcpServers(servers)`	Dynamically replace the set of MCP servers for this session. Returns info about which servers were added, removed, and any errors
`streamInput(stream)`	Stream input messages to the query for multi-turn conversations
`stopTask(taskId)`	Stop a running background task by ID
`close()`	Close the query and terminate the underlying process. Forcefully ends the query and cleans up all resources

`applyFlagSettings()`

Changes **settings** on a running session without restarting the query. Use it when a setting that has no dedicated setter needs to change mid-session, such as tightening `permissions` after the agent reads untrusted input. `setModel()` and `setPermissionMode()` are dedicated setters for those two keys; `applyFlagSettings()` is the general form that accepts any subset of the settings keys, and passing `model` here behaves the same as `setModel()`.

Only some keys take effect mid-session:

- **Applied on the next turn**: `model`, `effortLevel`, `ultracode`, `permissions`, `hooks`, `skillOverrides`, `fastMode`, `awaySummaryEnabled`, `agent`. Switching `agent` also applies

that agent's model override, hooks, and system prompt on the next turn.

- **No effect mid-session**: the system prompt options. These are resolved once at startup, so the running session keeps the original value even though the call succeeds. To change them, start a new session.

The values are written to the flag-settings layer, the same layer the inline `settings` option of `query()` populates at startup. Flag settings sit near the top of the **settings precedence order**: they override user, project, and local settings, and only managed policy settings can override them. This is the same tier the **on-page precedence section** calls programmatic options.

Successive calls shallow-merge top-level keys. A second call with `{ permissions: {...} }` replaces the entire `permissions` object from the prior call rather than deep-merging into it. To clear a key from the flag layer and fall back to lower-precedence sources, pass `null` for that key. Passing `undefined` has no effect because JSON serialization drops it.

Only available in streaming input mode, the same constraint as `setModel()` and `setPermissionMode()`.

The example below switches the active model mid-session, then clears the override so the model falls back to whatever the user or project settings specify.

```
const q = query({ prompt: messageStream });

// Override the model for the rest of the session
await q.applyFlagSettings({ model: "claude-opus-4-6" });

// Later: clear the override and fall back to lower-precedence
settings
await q.applyFlagSettings({ model: null });
```

`applyFlagSettings()` is TypeScript-only. The Python SDK does not expose an equivalent method.

`WarmQuery`

Handle returned by `startup()`. The subprocess is already spawned and initialized, so calling `query()` on this handle writes the prompt directly to a ready process with no startup latency.

```ts
interface WarmQuery extends AsyncDisposable {
  query(prompt: string | AsyncIterable<SDKUserMessage>): Query;
  close(): void;
}
```

Methods

Method	Description
`query(prompt)`	Send a prompt to the pre-warmed subprocess and return a `Query`. Can only be called once per `WarmQuery`
`close()`	Close the subprocess without sending a prompt. Use this to discard a warm query that is no longer needed

`WarmQuery` implements `AsyncDisposable`, so it can be used with `await using` for automatic cleanup.

`SDKControlInitializeResponse`

Return type of `initializationResult()`. Contains session initialization data.

```ts
type SDKControlInitializeResponse = {
  commands: SlashCommand[];
  agents: AgentInfo[];
  output_style: string;
  available_output_styles: string[];
  models: ModelInfo[];
  account: AccountInfo;
  fast_mode_state?: "off" | "cooldown" | "on";
};
```

When a client sends `initialize` to a session that is already running, the control-response wrapper also carries an optional `pending_permission_requests` array. The field is on the response wrapper itself, not in the `SDKControlInitializeResponse` payload above. Each entry is a complete `control_request` message with the same `{ type: "control_request", request_id, request }` shape the session streams for permission requests while running.

These are requests that were issued before the client connected and are still awaiting a reply, so read this array to surface in-flight permission prompts immediately; they will not be re-sent.

`AgentDefinition`

Configuration for a subagent defined programmatically.

```
type AgentDefinition = {
  description: string;
  tools?: string[];
  disallowedTools?: string[];
  prompt: string;
  model?: string;
  mcpServers?: AgentMcpServerSpec[];
  skills?: string[];
  initialPrompt?: string;
  maxTurns?: number;
  background?: boolean;
  memory?: "user" | "project" | "local";
  effort?: "low" | "medium" | "high" | "xhigh" | "max" | number;
  permissionMode?: PermissionMode;
  criticalSystemReminder_EXPERIMENTAL?: string;
};
```

Field	Required	Description
`description`	Yes	Natural language description of when to use this agent
`tools`	No	Array of allowed tool names. If omitted, inherits all tools from parent. To preload Skills into the agent's context, use the `skills` field rather than listing `'Skill'` here
`disallowedTools`	No	Array of tool names to explicitly disallow for this agent. MCP server-level patterns are also accepted: `mcp__server` or `mcp__server__*` removes every tool from that server, and `mcp__*` removes every MCP tool from any server
`prompt`	Yes	The agent's system prompt
`model`	No	Model override for this agent. Accepts an alias such as `'fable'`, `'opus'`, `'sonnet'`, `'haiku'`, `'inherit'`, or a full model ID. If omitted or `'inherit'`, uses the main model
`mcpServers`	No	MCP server specifications for this agent
`skills`	No	Array of skill names to preload into the agent context
`initialPrompt`	No	Auto-submitted as the first user turn when this agent runs as the main thread agent

Field	Required	Description
`maxTurns`	No	Maximum number of agentic turns (API round-trips) before stopping
`background`	No	Run this agent as a non-blocking background task when invoked
`memory`	No	Memory source for this agent: `'user'`, `'project'`, or `'local'`
`effort`	No	Reasoning effort level for this agent. Accepts a named level or an integer
`permissionMode`	No	Permission mode for tool execution within this agent. See `PermissionMode`
`criticalSystemReminder_EXPERIMENTAL`	No	Experimental: Critical reminder added to the system prompt

`AgentMcpServerSpec`

Specifies MCP servers available to a subagent. Can be a server name (string referencing a server from the parent's `mcpServers` config) or an inline server configuration record mapping server names to configs.

```
type AgentMcpServerSpec = string | Record<string,
McpServerConfigForProcessTransport>;
```

Where `McpServerConfigForProcessTransport` is `McpStdioServerConfig | McpSSEServerConfig | McpHttpServerConfig | McpSdkServerConfig`.

`SettingSource`

Controls which filesystem-based configuration sources the SDK loads settings from.

```
type SettingSource = "user" | "project" | "local";
```

Value	Description	Location
`'user'`	Global user settings	`~/.claude/settings.json`
`'project'`	Shared project settings (version controlled)	`.claude/settings.json`

Value	Description	Location
`'local'`	Local project settings (not version controlled)	`.claude/settings.local.json`

Default behavior

When `settingSources` is omitted or `undefined`, `query()` loads the same filesystem settings as the Claude Code CLI: user, project, and local. **Endpoint-managed policy** is loaded in all cases; server-managed settings are fetched when the session authenticates with an organization credential on an **eligible configuration**. See **What settingSources does not control** for inputs that are read regardless of this option, and how to disable them.

Why use settingSources

Disable filesystem settings:

```
// Do not load user, project, or local settings from disk
const result = query({
  prompt: "Analyze this code",
  options: { settingSources: [] }
});
```

Load all filesystem settings explicitly:

```
const result = query({
  prompt: "Analyze this code",
  options: {
    settingSources: ["user", "project", "local"] // Load all
settings
  }
});
```

Load only specific setting sources:

```typescript
// Load only project settings, ignore user and local
const result = query({
  prompt: "Run CI checks",
  options: {
    settingSources: ["project"] // Only .claude/settings.json
  }
});
```

Testing and CI environments:

```typescript
// Ensure consistent behavior in CI by excluding local settings
const result = query({
  prompt: "Run tests",
  options: {
    settingSources: ["project"], // Only team-shared settings
    permissionMode: "bypassPermissions"
  }
});
```

SDK-only applications:

```typescript
// Define everything programmatically.
// Pass [] to opt out of filesystem setting sources.
const result = query({
  prompt: "Review this PR",
  options: {
    settingSources: [],
    agents: {
      /* ... */
    },
    mcpServers: {
      /* ... */
    },
    allowedTools: ["Read", "Grep", "Glob"]
  }
});
```

Loading CLAUDE.md project instructions:

```
// Load project settings to include CLAUDE.md files
const result = query({
  prompt: "Add a new feature following project conventions",
  options: {
    systemPrompt: {
      type: "preset",
      preset: "claude_code" // Use Claude Code's system prompt
    },
    settingSources: ["project"], // Loads CLAUDE.md from project
directory
    allowedTools: ["Read", "Write", "Edit"]
  }
});
```

Settings precedence

When multiple sources are loaded, settings are merged with this precedence (highest to lowest):

1. Local settings (`.claude/settings.local.json`)
2. Project settings (`.claude/settings.json`)
3. User settings (`~/.claude/settings.json`)

Programmatic options such as `agents` , `allowedTools` , and `settings` override user, project, and local filesystem settings. Managed policy settings take precedence over programmatic options.

`PermissionMode`

```
type PermissionMode =
  | "default" // Standard permission behavior
  | "acceptEdits" // Auto-accept file edits
  | "bypassPermissions" // Bypass permission checks; explicit ask
rules still prompt
  | "plan" // Planning mode - explore without editing
  | "dontAsk" // Don't prompt for permissions, deny if not pre-
approved
  | "auto"; // Use a model classifier to approve or deny each
tool call
```

`CanUseTool`

Custom permission function type for controlling tool usage.

```
type CanUseTool = (
  toolName: string,
  input: Record<string, unknown>,
  options: {
    signal: AbortSignal;
    suggestions?: PermissionUpdate[];
    blockedPath?: string;
    decisionReason?: string;
    toolUseID: string;
    agentID?: string;
  }
) => Promise<PermissionResult>;
```

Option	Type	Description
`signal`	`AbortSignal`	Signaled if the operation should be aborted
`suggestions`	`PermissionUpdate[]`	Suggested permission updates so the user is not prompted again for this tool. Bash prompts include a suggestion with the `localSettings` **destination**, so returning it in `updatedPermissions` writes the rule to `.claude/settings.local.json` and persists across sessions.
`blockedPath`	`string`	The file path that triggered the permission request, if applicable
`decisionReason`	`string`	Explains why this permission request was triggered
`toolUseID`	`string`	Unique identifier for this specific tool call within the assistant message
`agentID`	`string`	If running within a sub-agent, the sub-agent's ID

`PermissionResult`

Result of a permission check.

```ts
type PermissionResult =
  | {
      behavior: "allow";
      updatedInput?: Record<string, unknown>;
      updatedPermissions?: PermissionUpdate[];
      toolUseID?: string;
    }
  | {
      behavior: "deny";
      message: string;
      interrupt?: boolean;
      toolUseID?: string;
    };
```

`ToolConfig`

Configuration for built-in tool behavior.

```ts
type ToolConfig = {
  askUserQuestion?: {
    previewFormat?: "markdown" | "html";
  };
};
```

Field	Type	Description
`askUserQuestion.previewFormat`	`'markdown' | 'html'`	Opts into the `preview` field on `AskUserQuestion` options and sets its content format. When unset, Claude does not emit previews

`McpServerConfig`

Configuration for MCP servers.

```ts
type McpServerConfig =
  | McpStdioServerConfig
  | McpSSEServerConfig
  | McpHttpServerConfig
  | McpSdkServerConfigWithInstance;
```

`McpStdioServerConfig`

```
type McpStdioServerConfig = {
  type?: "stdio";
  command: string;
  args?: string[];
  env?: Record<string, string>;
};
```

`McpSSEServerConfig`

```
type McpSSEServerConfig = {
  type: "sse";
  url: string;
  headers?: Record<string, string>;
};
```

`McpHttpServerConfig`

```
type McpHttpServerConfig = {
  type: "http";
  url: string;
  headers?: Record<string, string>;
};
```

`McpSdkServerConfigWithInstance`

```
type McpSdkServerConfigWithInstance = {
  type: "sdk";
  name: string;
  instance: McpServer;
};
```

`McpClaudeAIProxyServerConfig`

```
type McpClaudeAIProxyServerConfig = {
  type: "claudeai-proxy";
  url: string;
  id: string;
};
```

`SdkPluginConfig`

Configuration for loading plugins in the SDK.

```
type SdkPluginConfig = {
  type: "local";
  path: string;
  skipMcpDiscovery?: boolean;
};
```

Field	Type	Description
`type`	`'local'`	Must be `'local'` (only local plugins currently supported)
`path`	`string`	Absolute or relative path to the plugin directory
`skipMcpDiscovery`	`boolean`	When `true`, the SDK loads skills, hooks, agents, and commands from this plugin but does not read its `.mcp.json` or manifest `mcpServers`. Set this when your application owns the plugin's MCP connections.

Example:

```
plugins: [
  { type: "local", path: "./my-plugin" },
  { type: "local", path: "/absolute/path/to/plugin" }
];
```

For complete information on creating and using plugins, see **Plugins**.

Message Types

`SDKMessage`

Union type of all possible messages returned by the query.

```
type SDKMessage =
  | SDKAssistantMessage
  | SDKUserMessage
  | SDKUserMessageReplay
  | SDKResultMessage
  | SDKSystemMessage
  | SDKPartialAssistantMessage
  | SDKCompactBoundaryMessage
  | SDKStatusMessage
  | SDKLocalCommandOutputMessage
  | SDKHookStartedMessage
  | SDKHookProgressMessage
  | SDKHookResponseMessage
  | SDKPluginInstallMessage
  | SDKToolProgressMessage
  | SDKAuthStatusMessage
  | SDKTaskNotificationMessage
  | SDKTaskStartedMessage
  | SDKTaskProgressMessage
  | SDKTaskUpdatedMessage
  | SDKSessionStateChangedMessage
  | SDKWorkerShuttingDownMessage
  | SDKCommandsChangedMessage
  | SDKNotificationMessage
  | SDKFilesPersistedEvent
  | SDKToolUseSummaryMessage
  | SDKMemoryRecallMessage
  | SDKRateLimitEvent
  | SDKElicitationCompleteMessage
  | SDKPermissionDeniedMessage
  | SDKPromptSuggestionMessage
  | SDKAPIRetryMessage
  | SDKMirrorErrorMessage
  | SDKInformationalMessage;
```

`SDKAssistantMessage`

Assistant response message.

```
type SDKAssistantMessage = {
  type: "assistant";
  uuid: UUID;
  session_id: string;
  message: BetaMessage; // From Anthropic SDK
  parent_tool_use_id: string | null;
  error?: SDKAssistantMessageError;
};
```

The `message` field is a `BetaMessage` from the Anthropic SDK. It includes fields like `id`, `content`, `model`, `stop_reason`, and `usage`.

`SDKAssistantMessageError` is one of: `'authentication_failed'`, `'oauth_org_not_allowed'`, `'billing_error'`, `'rate_limit'`, `'overloaded'`, `'invalid_request'`, `'model_not_found'`, `'server_error'`, `'max_output_tokens'`, or `'unknown'`. `'model_not_found'` means the selected model doesn't exist or isn't available to your account or deployment. `'overloaded'` means the API returned a 529 because the server is at capacity, as opposed to `'rate_limit'`, which is a 429 against your quota.

`SDKUserMessage`

User input message.

```
type SDKUserMessage = {
  type: "user";
  uuid?: UUID;
  session_id?: string;
  message: MessageParam; // From Anthropic SDK
  parent_tool_use_id: string | null;
  isSynthetic?: boolean;
  shouldQuery?: boolean;
  tool_use_result?: unknown;
  origin?: SDKMessageOrigin;
};
```

Set `shouldQuery` to `false` to append the message to the transcript without triggering an

assistant turn. The message is held and merged into the next user message that does trigger a turn. Use this to inject context, such as the output of a command you ran out of band, without spending a model call on it.

`SDKUserMessageReplay`

Replayed user message with required UUID.

```
type SDKUserMessageReplay = {
  type: "user";
  uuid: UUID;
  session_id: string;
  message: MessageParam;
  parent_tool_use_id: string | null;
  isSynthetic?: boolean;
  tool_use_result?: unknown;
  origin?: SDKMessageOrigin;
  isReplay: true;
};
```

`SDKResultMessage`

Final result message.

```
type SDKResultMessage =
  | {
      type: "result";
      subtype: "success";
      uuid: UUID;
      session_id: string;
      duration_ms: number;
      duration_api_ms: number;
      is_error: boolean;
      api_error_status?: number | null;
      num_turns: number;
      result: string;
      stop_reason: string | null;
      ttft_ms?: number;
      ttft_stream_ms?: number;
      total_cost_usd: number;
      usage: NonNullableUsage;
      modelUsage: { [modelName: string]: ModelUsage };
      permission_denials: SDKPermissionDenial[];
      structured_output?: unknown;
      deferred_tool_use?: { id: string; name: string; input:
Record<string, unknown> };
      terminal_reason?: TerminalReason;
      fast_mode_state?: FastModeState;
      origin?: SDKMessageOrigin;
    }
  | {
      type: "result";
      subtype:
        | "error_max_turns"
        | "error_during_execution"
        | "error_max_budget_usd"
        | "error_max_structured_output_retries";
      uuid: UUID;
      session_id: string;
      duration_ms: number;
      duration_api_ms: number;
      is_error: boolean;
      num_turns: number;
      stop_reason: string | null;
      total_cost_usd: number;
      usage: NonNullableUsage;
      modelUsage: { [modelName: string]: ModelUsage };
      permission_denials: SDKPermissionDenial[];
```

```
    errors: string[];
    terminal_reason?: TerminalReason;
    fast_mode_state?: FastModeState;
    origin?: SDKMessageOrigin;
  };
```

Several fields on the result carry diagnostic detail beyond `subtype`:

- `api_error_status`: the HTTP status code of the API error that terminated the conversation. Absent or `null` when the turn ended without an API error.
- `ttft_ms`: time to first token in milliseconds, measured when the first complete assistant message arrives. Present on the success arm only.
- `ttft_stream_ms`: time in milliseconds until the first `message_start` stream event, when the response stream opens. Lower than `ttft_ms`; the gap between the two is time spent streaming the first message. Present on the success arm only.
- `terminal_reason`: why the loop ended. One of `"completed"`, `"max_turns"`, `"tool_deferred"`, `"aborted_streaming"`, `"aborted_tools"`, `"hook_stopped"`, `"stop_hook_prevented"`, `"blocking_limit"`, `"rapid_refill_breaker"`, `"prompt_too_long"`, `"image_error"`, or `"model_error"`.
- `fast_mode_state`: one of `"on"`, `"off"`, or `"cooldown"`.

The `origin` field forwards the `SDKMessageOrigin` of the user message that triggered this result. When a background task finishes and the SDK injects a synthetic follow-up turn, the resulting `SDKResultMessage` carries `origin: { kind: "task-notification" }`. Check this field to distinguish results that answer your prompt from results emitted for background-task follow-ups, so you can route or suppress the latter. The field is absent for results emitted before any user turn, such as startup errors.

When a `PreToolUse` hook returns `permissionDecision: "defer"`, the result has `stop_reason: "tool_deferred"` and `deferred_tool_use` carries the pending tool's `id`, `name`, and `input`. Read this field to surface the request in your own UI, then resume with the same `session_id` to continue. See **Defer a tool call for later** for the full round trip.

`SDKSystemMessage`

System initialization message.

```
type SDKSystemMessage = {
  type: "system";
  subtype: "init";
  uuid: UUID;
  session_id: string;
  agents?: string[];
  apiKeySource: ApiKeySource;
  betas?: string[];
  claude_code_version: string;
  cwd: string;
  tools: string[];
  mcp_servers: {
    name: string;
    status: string;
  }[];
  model: string;
  permissionMode: PermissionMode;
  slash_commands: string[];
  output_style: string;
  skills: string[];
  plugins: { name: string; path: string }[];
};
```

`SDKPartialAssistantMessage`

Streaming partial message (only when `includePartialMessages` is true).

```
type SDKPartialAssistantMessage = {
  type: "stream_event";
  event: BetaRawMessageStreamEvent; // From Anthropic SDK
  parent_tool_use_id: string | null;
  uuid: UUID;
  session_id: string;
  ttft_ms?: number; // Time to first token in ms, present only on
message_start events
};
```

`SDKCompactBoundaryMessage`

Message indicating a conversation compaction boundary.

```ts
type SDKCompactBoundaryMessage = {
  type: "system";
  subtype: "compact_boundary";
  uuid: UUID;
  session_id: string;
  compact_metadata: {
    trigger: "manual" | "auto";
    pre_tokens: number;
  };
};
```

`SDKInformationalMessage`

Generic text banner emitted by the loop. Carries non-error status lines, hook feedback such as a `UserPromptSubmit` hook's block reason, and command output. Render `content` as plaintext at the given `level`.

```ts
type SDKInformationalMessage = {
  type: "system";
  subtype: "informational";
  content: string;
  level: "info" | "notice" | "suggestion" | "warning";
  tool_use_id?: string;
  prevent_continuation?: boolean;
  uuid: UUID;
  session_id: string;
};
```

`SDKWorkerShuttingDownMessage`

Emitted on graceful worker teardown so remote clients can show why the worker exited instead of waiting for heartbeat timeout. The `reason` is a short snake_case string set by the host CLI, such as `"host_exit"` or `"remote_control_disabled"`. Act on this only when streaming live. A resumed session replays past instances of this message, so ignore them in that case.

```ts
type SDKWorkerShuttingDownMessage = {
  type: "system";
  subtype: "worker_shutting_down";
  reason: string;
  uuid: UUID;
  session_id: string;
};
```

`SDKPluginInstallMessage`

Plugin installation progress event. Emitted when `CLAUDE_CODE_SYNC_PLUGIN_INSTALL` is set, so your Agent SDK application can track marketplace plugin installation before the first turn. The `started` and `completed` statuses bracket the overall install. The `installed` and `failed` statuses report individual marketplaces and include `name`.

```ts
type SDKPluginInstallMessage = {
  type: "system";
  subtype: "plugin_install";
  status: "started" | "installed" | "failed" | "completed";
  name?: string;
  error?: string;
  uuid: UUID;
  session_id: string;
};
```

`SDKPermissionDeniedMessage`

Stream event emitted when the permission system auto-denies a tool call without an interactive prompt. Use it to render the denial in your UI as it happens, rather than only observing the `is_error` tool result that follows. The interactive ask path reaches your application separately through the `canUseTool` callback. Denials issued by a `PreToolUse` hook are not reported through this event.

This event requires Claude Code v2.1.136 or later.

```ts
type SDKPermissionDeniedMessage = {
  type: "system";
  subtype: "permission_denied";
  tool_name: string;
  tool_use_id: string;
  agent_id?: string;
  decision_reason_type?: string;
  decision_reason?: string;
  message: string;
  uuid: UUID;
  session_id: string;
};
```

Field	Type	Description
`tool_name`	`string`	Name of the tool that was denied
`tool_use_id`	`string`	ID of the `tool_use` block this denial answers
`agent_id`	`string`	Subagent ID when the denied call originated inside a subagent. Mirrors the field on `can_use_tool` for host-side routing
`decision_reason_type`	`string`	Discriminator for the component that decided, such as `"rule"`, `"mode"`, `"classifier"`, or `"asyncAgent"`
`decision_reason`	`string`	Human-readable reason from the deciding component, when available
`message`	`string`	Rejection message returned to the model in the `tool_result`

`SDKPermissionDenial`

Information about a denied tool use.

```ts
type SDKPermissionDenial = {
  tool_name: string;
  tool_use_id: string;
  tool_input: Record<string, unknown>;
};
```

`SDKMessageOrigin`

Provenance of a user-role message. This appears as `origin` on `SDKUserMessage` and is forwarded onto the corresponding `SDKResultMessage` so you can tell what triggered a given turn.

```
type SDKMessageOrigin =
  | { kind: "human" }
  | { kind: "channel"; server: string }
  | { kind: "peer"; from: string; name?: string; senderTaskId?:
string }
  | { kind: "task-notification" }
  | { kind: "coordinator" }
  | { kind: "auto-continuation" };
```

`kind`	Meaning
`human`	Direct input from the end user. On user messages, an absent `origin` also means human input.
`channel`	Message arriving on a **channel**. `server` is the source MCP server name.
`peer`	Message from another agent. For an in-process **teammate** sending to `main` via `SendMessage`, `from` is the teammate's name and `senderTaskId` is its task ID. For a cross-session peer such as another local Claude Code process, `from` is the sender address and `senderTaskId` is absent. The `name` field is reserved.
`task-notification`	Synthetic turn injected after a background task finished. See `SDKTaskNotificationMessage`.
`coordinator`	Message from a team coordinator in an **agent team**.
`auto-continuation`	Synthetic turn injected when the session continues without fresh user input, such as a command result that triggers a follow-up prompt.

Hook Types

For a comprehensive guide on using hooks with examples and common patterns, see the **Hooks guide**.

`HookEvent`

Available hook events.

```ts
type HookEvent =
  | "PreToolUse"
  | "PostToolUse"
  | "PostToolUseFailure"
  | "PostToolBatch"
  | "Notification"
  | "UserPromptSubmit"
  | "SessionStart"
  | "SessionEnd"
  | "Stop"
  | "SubagentStart"
  | "SubagentStop"
  | "PreCompact"
  | "PermissionRequest"
  | "Setup"
  | "TeammateIdle"
  | "TaskCompleted"
  | "ConfigChange"
  | "WorktreeCreate"
  | "WorktreeRemove"
  | "MessageDisplay";
```

`HookCallback`

Hook callback function type.

```ts
type HookCallback = (
  input: HookInput, // Union of all hook input types
  toolUseID: string | undefined,
  options: { signal: AbortSignal }
) => Promise<HookJSONOutput>;
```

`HookCallbackMatcher`

Hook configuration with optional matcher.

```
interface HookCallbackMatcher {
  matcher?: string;
  hooks: HookCallback[];
  timeout?: number; // Timeout in seconds for all hooks in this
matcher
}
```

`HookInput`

Union type of all hook input types.

```
type HookInput =
  | PreToolUseHookInput
  | PostToolUseHookInput
  | PostToolUseFailureHookInput
  | PostToolBatchHookInput
  | NotificationHookInput
  | UserPromptSubmitHookInput
  | SessionStartHookInput
  | SessionEndHookInput
  | StopHookInput
  | SubagentStartHookInput
  | SubagentStopHookInput
  | PreCompactHookInput
  | PermissionRequestHookInput
  | SetupHookInput
  | TeammateIdleHookInput
  | TaskCompletedHookInput
  | ConfigChangeHookInput
  | WorktreeCreateHookInput
  | WorktreeRemoveHookInput
  | MessageDisplayHookInput;
```

`BaseHookInput`

Base interface that all hook input types extend.

```ts
type BaseHookInput = {
  session_id: string;
  transcript_path: string;
  cwd: string;
  permission_mode?: string;
  effort?: { level: string };
  agent_id?: string;
  agent_type?: string;
};
```

`PreToolUseHookInput`

```ts
type PreToolUseHookInput = BaseHookInput & {
  hook_event_name: "PreToolUse";
  tool_name: string;
  tool_input: unknown;
  tool_use_id: string;
};
```

`PostToolUseHookInput`

```ts
type PostToolUseHookInput = BaseHookInput & {
  hook_event_name: "PostToolUse";
  tool_name: string;
  tool_input: unknown;
  tool_response: unknown;
  tool_use_id: string;
  duration_ms?: number;
};
```

`PostToolUseFailureHookInput`

```
type PostToolUseFailureHookInput = BaseHookInput & {
  hook_event_name: "PostToolUseFailure";
  tool_name: string;
  tool_input: unknown;
  tool_use_id: string;
  error: string;
  is_interrupt?: boolean;
  duration_ms?: number;
};
```

`PostToolBatchHookInput`

Fires once after every tool call in a batch has resolved, before the next model request. `tool_response` carries the serialized `tool_result` content the model sees; the shape differs from `PostToolUseHookInput`'s structured `Output` object.

```
type PostToolBatchHookInput = BaseHookInput & {
  hook_event_name: "PostToolBatch";
  tool_calls: PostToolBatchToolCall[];
};

type PostToolBatchToolCall = {
  tool_name: string;
  tool_input: unknown;
  tool_use_id: string;
  tool_response?: unknown;
};
```

`NotificationHookInput`

```
type NotificationHookInput = BaseHookInput & {
  hook_event_name: "Notification";
  message: string;
  title?: string;
  notification_type: string;
};
```

`UserPromptSubmitHookInput`

```
type UserPromptSubmitHookInput = BaseHookInput & {
  hook_event_name: "UserPromptSubmit";
  prompt: string;
};
```

`SessionStartHookInput`

```
type SessionStartHookInput = BaseHookInput & {
  hook_event_name: "SessionStart";
  source: "startup" | "resume" | "clear" | "compact";
  agent_type?: string;
  model?: string;
};
```

`SessionEndHookInput`

```
type SessionEndHookInput = BaseHookInput & {
  hook_event_name: "SessionEnd";
  reason: ExitReason; // String from EXIT_REASONS array
};
```

`StopHookInput`

```
type StopHookInput = BaseHookInput & {
  hook_event_name: "Stop";
  stop_hook_active: boolean;
  last_assistant_message?: string;
  background_tasks?: BackgroundTaskSummary[];
  session_crons?: SessionCronSummary[];
};
```

`SubagentStartHookInput`

```
type SubagentStartHookInput = BaseHookInput & {
  hook_event_name: "SubagentStart";
  agent_id: string;
  agent_type: string;
};
```

`SubagentStopHookInput`

```
type SubagentStopHookInput = BaseHookInput & {
  hook_event_name: "SubagentStop";
  stop_hook_active: boolean;
  agent_id: string;
  agent_transcript_path: string;
  agent_type: string;
  last_assistant_message?: string;
  background_tasks?: BackgroundTaskSummary[];
  session_crons?: SessionCronSummary[];
};

type BackgroundTaskSummary = {
  id: string;
  type: string;
  status: string;
  description: string;
  command?: string;
  agent_type?: string;
  server?: string;
  tool?: string;
  name?: string;
};

type SessionCronSummary = {
  id: string;
  schedule: string;
  recurring: boolean;
  prompt: string;
};
```

PreCompactHookInput

```
type PreCompactHookInput = BaseHookInput & {
  hook_event_name: "PreCompact";
  trigger: "manual" | "auto";
  custom_instructions: string | null;
};
```

PermissionRequestHookInput

```
type PermissionRequestHookInput = BaseHookInput & {
  hook_event_name: "PermissionRequest";
  tool_name: string;
  tool_input: unknown;
  permission_suggestions?: PermissionUpdate[];
};
```

SetupHookInput

```
type SetupHookInput = BaseHookInput & {
  hook_event_name: "Setup";
  trigger: "init" | "maintenance";
};
```

TeammateIdleHookInput

```
type TeammateIdleHookInput = BaseHookInput & {
  hook_event_name: "TeammateIdle";
  teammate_name: string;
  /** @deprecated since v2.1.178. Carries the session-derived
team name; will be removed. */
  team_name: string;
};
```

`TaskCompletedHookInput`

```
type TaskCompletedHookInput = BaseHookInput & {
  hook_event_name: "TaskCompleted";
  task_id: string;
  task_subject: string;
  task_description?: string;
  teammate_name?: string;
  /** @deprecated since v2.1.178. Carries the session-derived
team name; will be removed. */
  team_name?: string;
};
```

`ConfigChangeHookInput`

```
type ConfigChangeHookInput = BaseHookInput & {
  hook_event_name: "ConfigChange";
  source:
    | "user_settings"
    | "project_settings"
    | "local_settings"
    | "policy_settings"
    | "skills";
  file_path?: string;
};
```

`WorktreeCreateHookInput`

```
type WorktreeCreateHookInput = BaseHookInput & {
  hook_event_name: "WorktreeCreate";
  name: string;
};
```

`WorktreeRemoveHookInput`

```ts
type WorktreeRemoveHookInput = BaseHookInput & {
  hook_event_name: "WorktreeRemove";
  worktree_path: string;
};
```

`MessageDisplayHookInput`

```ts
type MessageDisplayHookInput = BaseHookInput & {
  hook_event_name: "MessageDisplay";
  turn_id: string;
  message_id: string;
  index: number;
  final: boolean;
  delta: string;
};
```

`HookJSONOutput`

Hook return value.

```ts
type HookJSONOutput = AsyncHookJSONOutput | SyncHookJSONOutput;
```

`AsyncHookJSONOutput`

```ts
type AsyncHookJSONOutput = {
  async: true;
  asyncTimeout?: number;
};
```

SyncHookJSONOutput

```
type SyncHookJSONOutput = {
  continue?: boolean;
  suppressOutput?: boolean;
  stopReason?: string;
  decision?: "approve" | "block";
  systemMessage?: string;
  reason?: string;
  hookSpecificOutput?:
    | {
        hookEventName: "PreToolUse";
        permissionDecision?: "allow" | "deny" | "ask" | "defer";
        permissionDecisionReason?: string;
        updatedInput?: Record<string, unknown>;
        additionalContext?: string;
      }
    | {
        hookEventName: "UserPromptSubmit";
        additionalContext?: string;
      }
    | {
        hookEventName: "SessionStart";
        additionalContext?: string;
      }
    | {
        hookEventName: "Setup";
        additionalContext?: string;
      }
    | {
        hookEventName: "SubagentStart";
        additionalContext?: string;
      }
    | {
        hookEventName: "PostToolUse";
        additionalContext?: string;
        updatedToolOutput?: unknown;
        /** @deprecated Use `updatedToolOutput`, which works for
all tools. */
        updatedMCPToolOutput?: unknown;
      }
    | {
        hookEventName: "PostToolUseFailure";
        additionalContext?: string;
```

```
      }
    | {
        hookEventName: "PostToolBatch";
        additionalContext?: string;
      }
    | {
        hookEventName: "Notification";
        additionalContext?: string;
      }
    | {
        hookEventName: "PermissionRequest";
        decision:
          | {
              behavior: "allow";
              updatedInput?: Record<string, unknown>;
              updatedPermissions?: PermissionUpdate[];
            }
          | {
              behavior: "deny";
              message?: string;
              interrupt?: boolean;
            };
      };
};
```

Tool Input Types

Documentation of input schemas for all built-in Claude Code tools. These types are exported from `@anthropic-ai/claude-agent-sdk` and can be used for type-safe tool interactions.

`ToolInputSchemas`

Union of all tool input types, exported from `@anthropic-ai/claude-agent-sdk`.

```
type ToolInputSchemas =
  | AgentInput
  | AskUserQuestionInput
  | BashInput
  | TaskOutputInput
  | EnterWorktreeInput
  | ExitPlanModeInput
  | FileEditInput
  | FileReadInput
  | FileWriteInput
  | GlobInput
  | GrepInput
  | ListMcpResourcesInput
  | McpInput
  | MonitorInput
  | NotebookEditInput
  | ReadMcpResourceInput
  | SubscribeMcpResourceInput
  | SubscribePollingInput
  | TaskCreateInput
  | TaskGetInput
  | TaskListInput
  | TaskStopInput
  | TaskUpdateInput
  | TodoWriteInput
  | UnsubscribeMcpResourceInput
  | UnsubscribePollingInput
  | WebFetchInput
  | WebSearchInput
  | WorkflowInput;
```

Agent

Tool name: `Agent` (previously `Task`, which is still accepted as an alias)

```ts
type AgentInput = {
  description: string;
  prompt: string;
  subagent_type: string;
  model?: "sonnet" | "opus" | "haiku" | "fable";
  resume?: string;
  run_in_background?: boolean;
  max_turns?: number;
  name?: string;
  mode?: "acceptEdits" | "bypassPermissions" | "default" |
"dontAsk" | "plan";
  isolation?: "worktree";
};
```

Launches a new agent to handle complex, multi-step tasks autonomously.

AskUserQuestion

Tool name: `AskUserQuestion`

```ts
type AskUserQuestionInput = {
  questions: Array<{
    question: string;
    header: string;
    options: Array<{ label: string; description: string;
preview?: string }>;
    multiSelect: boolean;
  }>;
};
```

Asks the user clarifying questions during execution. See **Handle approvals and user input** for usage details.

Bash

Tool name: `Bash`

```ts
type BashInput = {
  command: string;
  timeout?: number;
  description?: string;
  run_in_background?: boolean;
  dangerouslyDisableSandbox?: boolean;
};
```

Executes bash commands in a persistent shell session with optional timeout and background execution.

Monitor

Tool name: `Monitor`

```ts
type MonitorInput = {
  command: string;
  description: string;
  timeout_ms?: number;
  persistent?: boolean;
};
```

Runs a background script and delivers each stdout line to Claude as an event so it can react without polling. Set `persistent: true` for session-length watches such as log tails. Monitor follows the same permission rules as Bash. See the **Monitor tool reference** for behavior and provider availability.

TaskOutput

Tool name: `TaskOutput`

```ts
type TaskOutputInput = {
  task_id: string;
  block: boolean;
  timeout: number;
};
```

Retrieves output from a running or completed background task.

Edit

Tool name: `Edit`

```
type FileEditInput = {
  file_path: string;
  old_string: string;
  new_string: string;
  replace_all?: boolean;
};
```

Performs exact string replacements in files.

Read

Tool name: `Read`

```
type FileReadInput = {
  file_path: string;
  offset?: number;
  limit?: number;
  pages?: string;
};
```

Reads files from the local filesystem, including text, images, PDFs, and Jupyter notebooks. Use `pages` for PDF page ranges (for example, `"1-5"`).

Write

Tool name: `Write`

```
type FileWriteInput = {
  file_path: string;
  content: string;
};
```

Writes a file to the local filesystem, overwriting if it exists.

Glob

Tool name: `Glob`

```
type GlobInput = {
  pattern: string;
  path?: string;
};
```

Fast file pattern matching that works with any codebase size.

Grep

Tool name: `Grep`

```
type GrepInput = {
  pattern: string;
  path?: string;
  glob?: string;
  type?: string;
  output_mode?: "content" | "files_with_matches" | "count";
  "-i"?: boolean;
  "-n"?: boolean;
  "-B"?: number;
  "-A"?: number;
  "-C"?: number;
  context?: number;
  head_limit?: number;
  offset?: number;
  multiline?: boolean;
};
```

Powerful search tool built on ripgrep with regex support.

TaskStop

Tool name: `TaskStop`

```ts
type TaskStopInput = {
  task_id?: string;
  shell_id?: string; // Deprecated: use task_id
};
```

Stops a running background task or shell by ID.

NotebookEdit

Tool name: `NotebookEdit`

```ts
type NotebookEditInput = {
  notebook_path: string;
  cell_id?: string;
  new_source: string;
  cell_type?: "code" | "markdown";
  edit_mode?: "replace" | "insert" | "delete";
};
```

Edits cells in Jupyter notebook files.

WebFetch

Tool name: `WebFetch`

```ts
type WebFetchInput = {
  url: string;
  prompt: string;
};
```

Fetches content from a URL and processes it with an AI model.

WebSearch

Tool name: `WebSearch`

```ts
type WebSearchInput = {
  query: string;
  allowed_domains?: string[];
  blocked_domains?: string[];
};
```

Searches the web and returns formatted results.

Workflow

Tool name: `Workflow`

```ts
type WorkflowInput = {
  script?: string;
  name?: string;
  scriptPath?: string;
  args?: unknown;
  resumeFromRunId?: string;
};
```

Runs a **dynamic workflow**: a script that orchestrates many subagents in the background and returns one consolidated result. The `Workflow` tool is available in Agent SDK v0.3.149 and later. At least one of `script`, `name`, or `scriptPath` is required.

Field	Type	Description
`script`	`string`	Inline workflow script. Must begin with `export const meta = { name, description }` as a literal, followed by the script body using `agent()`, `parallel()`, `pipeline()`, and `phase()`. An optional `phases` array in `meta` groups agents under named stages in the progress view
`name`	`string`	Name of a built-in workflow or one saved in `.claude/workflows/`. Resolved to a script
`scriptPath`	`string`	Path to a workflow script file on disk. Takes precedence over `script` and `name`. Every invocation persists its script and returns the path in the result, so you can edit that file and re-invoke with the same `scriptPath` to iterate

Field	Type	Description
`args`	`unknown`	Input value exposed to the script as the global `args`, for parameterized named workflows such as a research question or a list of file paths. Pass arrays and objects as actual JSON values, not as a JSON-encoded string
`resumeFromRunId`	`string`	Run ID of a prior `Workflow` invocation to resume. Completed `agent()` calls with unchanged inputs return cached results; only changed or new calls run live. Same session only

TodoWrite

Tool name: `TodoWrite`

```
type TodoWriteInput = {
  todos: Array<{
    content: string;
    status: "pending" | "in_progress" | "completed";
    activeForm: string;
  }>;
};
```

Creates and manages a structured task list for tracking progress.

> As of TypeScript Agent SDK 0.3.142, `TodoWrite` is disabled by default. Use `TaskCreate`, `TaskGet`, `TaskUpdate`, and `TaskList` instead. See **Migrate to Task tools** to update your monitoring code, or set `CLAUDE_CODE_ENABLE_TASKS=0` to revert to `TodoWrite`.

TaskCreate

Tool name: `TaskCreate`

```
type TaskCreateInput = {
  subject: string;
  description: string;
  activeForm?: string;
  metadata?: Record<string, unknown>;
};
```

Creates a single task and returns its assigned ID.

TaskUpdate

Tool name: `TaskUpdate`

```ts
type TaskUpdateInput = {
  taskId: string;
  status?: "pending" | "in_progress" | "completed" | "deleted";
  subject?: string;
  description?: string;
  activeForm?: string;
  addBlocks?: string[];
  addBlockedBy?: string[];
  owner?: string;
  metadata?: Record<string, unknown>;
};
```

Patches one task by ID. Set `status` to `"deleted"` to remove it.

TaskGet

Tool name: `TaskGet`

```ts
type TaskGetInput = {
  taskId: string;
};
```

Returns full details for one task, or `null` when the ID is not found.

TaskList

Tool name: `TaskList`

```ts
type TaskListInput = {};
```

Returns a snapshot of all tasks in the current list.

ExitPlanMode

Tool name: `ExitPlanMode`

```
type ExitPlanModeInput = {
  allowedPrompts?: Array<{
    tool: "Bash";
    prompt: string;
  }>;
};
```

Exits planning mode. Optionally specifies prompt-based permissions needed to implement the plan.

ListMcpResources

Tool name: `ListMcpResourcesTool`

```
type ListMcpResourcesInput = {
  server?: string;
};
```

Lists available MCP resources from connected servers.

ReadMcpResource

Tool name: `ReadMcpResourceTool`

```
type ReadMcpResourceInput = {
  server: string;
  uri: string;
};
```

Reads a specific MCP resource from a server.

EnterWorktree

Tool name: `EnterWorktree`

```ts
type EnterWorktreeInput = {
  name?: string;
  path?: string;
};
```

Creates and enters a temporary git worktree for isolated work. Pass `path` to switch into an existing worktree of the current repository instead of creating a new one. `name` and `path` are mutually exclusive.

Tool Output Types

Documentation of output schemas for all built-in Claude Code tools. These types are exported from `@anthropic-ai/claude-agent-sdk` and represent the actual response data returned by each tool.

`ToolOutputSchemas`

Union of all tool output types.

```
type ToolOutputSchemas =
  | AgentOutput
  | AskUserQuestionOutput
  | BashOutput
  | EnterWorktreeOutput
  | ExitPlanModeOutput
  | FileEditOutput
  | FileReadOutput
  | FileWriteOutput
  | GlobOutput
  | GrepOutput
  | ListMcpResourcesOutput
  | MonitorOutput
  | NotebookEditOutput
  | ReadMcpResourceOutput
  | TaskCreateOutput
  | TaskGetOutput
  | TaskListOutput
  | TaskStopOutput
  | TaskUpdateOutput
  | TodoWriteOutput
  | WebFetchOutput
  | WebSearchOutput
  | WorkflowOutput;
```

Agent

Tool name: `Agent` (previously `Task`, which is still accepted as an alias)

```ts
type AgentOutput =
  | {
      status: "completed";
      agentId: string;
      content: Array<{ type: "text"; text: string }>;
      resolvedModel?: string;
      totalToolUseCount: number;
      totalDurationMs: number;
      totalTokens: number;
      usage: {
        input_tokens: number;
        output_tokens: number;
        cache_creation_input_tokens: number | null;
        cache_read_input_tokens: number | null;
        server_tool_use: {
          web_search_requests: number;
          web_fetch_requests: number;
        } | null;
        service_tier: ("standard" | "priority" | "batch") | null;
        cache_creation: {
          ephemeral_1h_input_tokens: number;
          ephemeral_5m_input_tokens: number;
        } | null;
      };
      prompt: string;
    }
  | {
      status: "async_launched";
      agentId: string;
      description: string;
      resolvedModel?: string;
      prompt: string;
      outputFile: string;
      canReadOutputFile?: boolean;
    }
  | {
      status: "sub_agent_entered";
      description: string;
      message: string;
    };
```

Returns the result from the subagent. Discriminated on the `status` field: `"completed"` for finished tasks, `"async_launched"` for background tasks, and `"sub_agent_entered"` for interactive

subagents.

The `resolvedModel` field on the `completed` and `async_launched` variants names the model the subagent actually ran on, which can differ from the requested `model` input when `availableModels` or another override applies. This field requires Claude Code v2.1.174 or later.

AskUserQuestion

Tool name: `AskUserQuestion`

```
type AskUserQuestionOutput = {
  questions: Array<{
    question: string;
    header: string;
    options: Array<{ label: string; description: string;
preview?: string }>;
    multiSelect: boolean;
  }>;
  answers: Record<string, string>;
  response?: string;
};
```

Returns the questions asked and the user's answers. `response` is set when the user typed a freeform reply instead of answering the structured questions; when present, Claude receives "The user responded: …" instead of the per-question answer list.

Bash

Tool name: `Bash`

```ts
type BashOutput = {
  stdout: string;
  stderr: string;
  rawOutputPath?: string;
  interrupted: boolean;
  isImage?: boolean;
  backgroundTaskId?: string;
  backgroundedByUser?: boolean;
  dangerouslyDisableSandbox?: boolean;
  returnCodeInterpretation?: string;
  structuredContent?: unknown[];
  persistedOutputPath?: string;
  persistedOutputSize?: number;
};
```

Returns command output with stdout/stderr split. Background commands include a `backgroundTaskId`.

Monitor

Tool name: `Monitor`

```ts
type MonitorOutput = {
  taskId: string;
  timeoutMs: number;
  persistent?: boolean;
};
```

Returns the background task ID for the running monitor. Use this ID with `TaskStop` to cancel the watch early.

Edit

Tool name: `Edit`

```typescript
type FileEditOutput = {
  filePath: string;
  oldString: string;
  newString: string;
  originalFile: string;
  structuredPatch: Array<{
    oldStart: number;
    oldLines: number;
    newStart: number;
    newLines: number;
    lines: string[];
  }>;
  userModified: boolean;
  replaceAll: boolean;
  gitDiff?: {
    filename: string;
    status: "modified" | "added";
    additions: number;
    deletions: number;
    changes: number;
    patch: string;
  };
};
```

Returns the structured diff of the edit operation.

Read

Tool name: `Read`

```
type FileReadOutput =
  | {
      type: "text";
      file: {
        filePath: string;
        content: string;
        numLines: number;
        startLine: number;
        totalLines: number;
      };
    }
  | {
      type: "image";
      file: {
        base64: string;
        type: "image/jpeg" | "image/png" | "image/gif" |
"image/webp";
        originalSize: number;
        dimensions?: {
          originalWidth?: number;
          originalHeight?: number;
          displayWidth?: number;
          displayHeight?: number;
        };
      };
    }
  | {
      type: "notebook";
      file: {
        filePath: string;
        cells: unknown[];
      };
    }
  | {
      type: "pdf";
      file: {
        filePath: string;
        base64: string;
        originalSize: number;
      };
    }
  | {
      type: "parts";
      file: {
```

```
      filePath: string;
      originalSize: number;
      count: number;
      outputDir: string;
    };
  };
```

Returns file contents in a format appropriate to the file type. Discriminated on the `type` field.

Write

Tool name: `Write`

```
type FileWriteOutput = {
  type: "create" | "update";
  filePath: string;
  content: string;
  structuredPatch: Array<{
    oldStart: number;
    oldLines: number;
    newStart: number;
    newLines: number;
    lines: string[];
  }>;
  originalFile: string | null;
  gitDiff?: {
    filename: string;
    status: "modified" | "added";
    additions: number;
    deletions: number;
    changes: number;
    patch: string;
  };
};
```

Returns the write result with structured diff information.

Glob

Tool name: `Glob`

```ts
type GlobOutput = {
  durationMs: number;
  numFiles: number;
  filenames: string[];
  truncated: boolean;
};
```

Returns file paths matching the glob pattern, sorted by modification time.

Grep

Tool name: `Grep`

```ts
type GrepOutput = {
  mode?: "content" | "files_with_matches" | "count";
  numFiles: number;
  filenames: string[];
  content?: string;
  numLines?: number;
  numMatches?: number;
  appliedLimit?: number;
  appliedOffset?: number;
};
```

Returns search results. The shape varies by `mode`: file list, content with matches, or match counts.

TaskStop

Tool name: `TaskStop`

```ts
type TaskStopOutput = {
  message: string;
  task_id: string;
  task_type: string;
  command?: string;
};
```

Returns confirmation after stopping the background task.

NotebookEdit

Tool name: `NotebookEdit`

```
type NotebookEditOutput = {
  new_source: string;
  cell_id?: string;
  cell_type: "code" | "markdown";
  language: string;
  edit_mode: string;
  error?: string;
  notebook_path: string;
  original_file: string;
  updated_file: string;
};
```

Returns the result of the notebook edit with original and updated file contents.

WebFetch

Tool name: `WebFetch`

```
type WebFetchOutput = {
  bytes: number;
  code: number;
  codeText: string;
  result: string;
  durationMs: number;
  url: string;
};
```

Returns the fetched content with HTTP status and metadata.

WebSearch

Tool name: `WebSearch`

```
type WebSearchOutput = {
  query: string;
  results: Array<
    | {
        tool_use_id: string;
        content: Array<{ title: string; url: string }>;
      }
    | string
  >;
  durationSeconds: number;
};
```

Returns search results from the web.

Workflow

Tool name: `Workflow`

```
type WorkflowOutput = {
  status: "async_launched";
  taskId: string;
  runId?: string;
  summary?: string;
  transcriptDir?: string;
  scriptPath?: string;
  error?: string;
};
```

Returns immediately after the tool accepts the invocation. The final result arrives later as a task completion. Check `error` before treating the run as started: a script that fails its syntax check returns `status: "async_launched"` with `error` set, and never runs.

Field	Type	Description
`status`	`"async_launched"`	The tool accepted the invocation. This is the only value the field takes
`taskId`	`string`	Background task identifier for the run
`runId`	`string`	Workflow run identifier to pass as `resumeFromRunId` on a later invocation

Field	Type	Description
`summary`	`string`	One-line description of what the workflow does
`transcriptDir`	`string`	Directory where subagent transcripts are written during execution
`scriptPath`	`string`	Path to the persisted workflow script for this run. Edit it and pass back as `scriptPath` to rerun without resending the script
`error`	`string`	Set when the script fails its syntax check. When present, the run did not start despite the `async_launched` status

TodoWrite

Tool name: `TodoWrite`

```
type TodoWriteOutput = {
  oldTodos: Array<{
    content: string;
    status: "pending" | "in_progress" | "completed";
    activeForm: string;
  }>;
  newTodos: Array<{
    content: string;
    status: "pending" | "in_progress" | "completed";
    activeForm: string;
  }>;
};
```

Returns the previous and updated task lists.

> As of TypeScript Agent SDK 0.3.142, `TodoWrite` is disabled by default. Use `TaskCreate`, `TaskGet`, `TaskUpdate`, and `TaskList` instead. See **Migrate to Task tools** to update your monitoring code, or set `CLAUDE_CODE_ENABLE_TASKS=0` to revert to `TodoWrite`.

TaskCreate

Tool name: `TaskCreate`

```
type TaskCreateOutput = {
  task: {
    id: string;
    subject: string;
  };
};
```

Returns the created task with its assigned ID.

TaskUpdate

Tool name: `TaskUpdate`

```
type TaskUpdateOutput = {
  success: boolean;
  taskId: string;
  updatedFields: string[];
  error?: string;
  statusChange?: {
    from: string;
    to: string;
  };
};
```

Returns the update result, including which fields changed.

TaskGet

Tool name: `TaskGet`

```
type TaskGetOutput = {
  task: {
    id: string;
    subject: string;
    description: string;
    status: "pending" | "in_progress" | "completed";
    blocks: string[];
    blockedBy: string[];
  } | null;
};
```

Returns the full task record, or `null` when the ID is not found.

TaskList

Tool name: `TaskList`

```
type TaskListOutput = {
  tasks: Array<{
    id: string;
    subject: string;
    status: "pending" | "in_progress" | "completed";
    owner?: string;
    blockedBy: string[];
  }>;
};
```

Returns a snapshot of all tasks in the current list.

ExitPlanMode

Tool name: `ExitPlanMode`

```
type ExitPlanModeOutput = {
  plan: string | null;
  isAgent: boolean;
  filePath?: string;
  hasTaskTool?: boolean;
  awaitingLeaderApproval?: boolean;
  requestId?: string;
};
```

Returns the plan state after exiting plan mode.

ListMcpResources

Tool name: `ListMcpResourcesTool`

```
type ListMcpResourcesOutput = Array<{
  uri: string;
  name: string;
  mimeType?: string;
  description?: string;
  server: string;
}>;
```

Returns an array of available MCP resources.

ReadMcpResource

Tool name: `ReadMcpResourceTool`

```
type ReadMcpResourceOutput = {
  contents: Array<{
    uri: string;
    mimeType?: string;
    text?: string;
  }>;
};
```

Returns the contents of the requested MCP resource.

EnterWorktree

Tool name: `EnterWorktree`

```
type EnterWorktreeOutput = {
  worktreePath: string;
  worktreeBranch?: string;
  message: string;
};
```

Returns information about the git worktree.

Permission Types

`PermissionUpdate`

Operations for updating permissions.

```ts
type PermissionUpdate =
  | {
      type: "addRules";
      rules: PermissionRuleValue[];
      behavior: PermissionBehavior;
      destination: PermissionUpdateDestination;
    }
  | {
      type: "replaceRules";
      rules: PermissionRuleValue[];
      behavior: PermissionBehavior;
      destination: PermissionUpdateDestination;
    }
  | {
      type: "removeRules";
      rules: PermissionRuleValue[];
      behavior: PermissionBehavior;
      destination: PermissionUpdateDestination;
    }
  | {
      type: "setMode";
      mode: PermissionMode;
      destination: PermissionUpdateDestination;
    }
  | {
      type: "addDirectories";
      directories: string[];
      destination: PermissionUpdateDestination;
    }
  | {
      type: "removeDirectories";
      directories: string[];
      destination: PermissionUpdateDestination;
    };
```

`PermissionBehavior`

```ts
type PermissionBehavior = "allow" | "deny" | "ask";
```

`PermissionUpdateDestination`

```
type PermissionUpdateDestination =
  | "userSettings" // Global user settings
  | "projectSettings" // Per-directory project settings
  | "localSettings" // Local project settings
  | "session" // Current session only
  | "cliArg"; // CLI argument
```

`PermissionRuleValue`

```
type PermissionRuleValue = {
  toolName: string;
  ruleContent?: string;
};
```

Other Types

`ApiKeySource`

```
type ApiKeySource = "user" | "project" | "org" | "temporary" |
"oauth";
```

`SdkBeta`

Available beta features that can be enabled via the `betas` option. See **Beta headers** for more information.

```
type SdkBeta = "context-1m-2025-08-07";
```

⚠ The `context-1m-2025-08-07` beta is retired as of April 30, 2026. Passing this value with Claude Sonnet 4.5 or Sonnet 4 has no effect, and requests that exceed the standard 200k-token context window return an error. To use a 1M-token context window, migrate to **Claude Sonnet 4.6, Claude Opus 4.6, Claude Opus 4.7, or Claude Opus 4.8**, which include 1M context at standard pricing with no beta header required.

`SlashCommand`

Information about an available slash command.

```
type SlashCommand = {
  name: string;
  description: string;
  argumentHint: string;
  aliases?: string[];
};
```

`ModelInfo`

Information about an available model.

```
type ModelInfo = {
  value: string;
  displayName: string;
  description: string;
  supportsEffort?: boolean;
  supportedEffortLevels?: ("low" | "medium" | "high" | "xhigh" |
"max")[];
  supportsAdaptiveThinking?: boolean;
  supportsFastMode?: boolean;
};
```

`AgentInfo`

Information about an available subagent that can be invoked via the Agent tool.

```
type AgentInfo = {
  name: string;
  description: string;
  model?: string;
};
```

Field	Type	Description
`name`	`string`	Agent type identifier (e.g., `"Explore"`, `"general-purpose"`)

Field	Type	Description
`description`	`string`	Description of when to use this agent
`model`	`string | undefined`	Model alias this agent uses. If omitted, inherits the parent's model

`McpServerStatus`

Status of a connected MCP server.

```
type McpServerStatus = {
  name: string;
  status: "connected" | "failed" | "needs-auth" | "pending" |
"disabled";
  serverInfo?: {
    name: string;
    version: string;
  };
  error?: string;
  config?: McpServerStatusConfig;
  scope?: string;
  tools?: {
    name: string;
    description?: string;
    annotations?: {
      readOnly?: boolean;
      destructive?: boolean;
      openWorld?: boolean;
    };
  }[];
};
```

`McpServerStatusConfig`

The configuration of an MCP server as reported by `mcpServerStatus()`. This is the union of all MCP server transport types.

```
type McpServerStatusConfig =
  | McpStdioServerConfig
  | McpSSEServerConfig
  | McpHttpServerConfig
  | McpSdkServerConfig
  | McpClaudeAIProxyServerConfig;
```

See `McpServerConfig` for details on each transport type.

`AccountInfo`

Account information for the authenticated user.

```
type AccountInfo = {
  email?: string;
  organization?: string;
  subscriptionType?: string;
  tokenSource?: string;
  apiKeySource?: string;
};
```

`ModelUsage`

Per-model usage statistics returned in result messages. The `costUSD` value is a client-side estimate. See **Track cost and usage** for billing caveats.

```
type ModelUsage = {
  inputTokens: number;
  outputTokens: number;
  cacheReadInputTokens: number;
  cacheCreationInputTokens: number;
  webSearchRequests: number;
  costUSD: number;
  contextWindow: number;
  maxOutputTokens: number;
};
```

`ConfigScope`

```
type ConfigScope = "local" | "user" | "project";
```

`NonNullableUsage`

A version of `Usage` with all nullable fields made non-nullable.

```
type NonNullableUsage = {
  [K in keyof Usage]: NonNullable<Usage[K]>;
};
```

`Usage`

Token usage statistics. This is the `BetaUsage` type from `@anthropic-ai/sdk`.

```
type Usage = {
  input_tokens: number;
  output_tokens: number;
  cache_creation_input_tokens: number | null;
  cache_read_input_tokens: number | null;
  cache_creation: {
    ephemeral_5m_input_tokens: number;
    ephemeral_1h_input_tokens: number;
  } | null;
  server_tool_use: BetaServerToolUsage | null;
  service_tier: "standard" | "priority" | "batch" | null;
  speed: "standard" | "fast" | null;
  inference_geo: string | null;
  iterations: BetaIterationsUsage | null;
};
```

`BetaServerToolUsage` and `BetaIterationsUsage` are defined in `@anthropic-ai/sdk`.

`CallToolResult`

MCP tool result type (from `@modelcontextprotocol/sdk/types.js`). `structuredContent` is a JSON object that can be returned alongside `content`, including image blocks. See **Return**

structured data.

```
type CallToolResult = {
  content: Array<{
    type: "text" | "image" | "audio" | "resource" |
"resource_link";
    // Additional fields vary by type
  }>;
  structuredContent?: Record<string, unknown>;
  isError?: boolean;
};
```

`ThinkingConfig`

Controls Claude's thinking/reasoning behavior. Takes precedence over the deprecated `maxThinkingTokens`.

```
type ThinkingDisplay = "summarized" | "omitted";

type ThinkingConfig =
  | { type: "adaptive"; display?: ThinkingDisplay } // The model
determines when and how much to reason (Opus 4.6+)
  | { type: "enabled"; budgetTokens?: number; display?:
ThinkingDisplay } // Fixed thinking token budget
  | { type: "disabled" }; // No extended thinking
```

The optional `display` field controls whether thinking text is returned `"summarized"` or `"omitted"`. On Claude Opus 4.7 and later, the API default is `"omitted"`, so set `"summarized"` to receive thinking content in `thinking` blocks.

`SpawnedProcess`

Interface for custom process spawning (used with `spawnClaudeCodeProcess` option). `ChildProcess` already satisfies this interface.

```
interface SpawnedProcess {
  stdin: Writable;
  stdout: Readable;
  readonly killed: boolean;
  readonly exitCode: number | null;
  kill(signal: NodeJS.Signals): boolean;
  on(
    event: "exit",
    listener: (code: number | null, signal: NodeJS.Signals |
null) => void
  ): void;
  on(event: "error", listener: (error: Error) => void): void;
  once(
    event: "exit",
    listener: (code: number | null, signal: NodeJS.Signals |
null) => void
  ): void;
  once(event: "error", listener: (error: Error) => void): void;
  off(
    event: "exit",
    listener: (code: number | null, signal: NodeJS.Signals |
null) => void
  ): void;
  off(event: "error", listener: (error: Error) => void): void;
}
```

`SpawnOptions`

Options passed to the custom spawn function.

```
interface SpawnOptions {
  command: string;
  args: string[];
  cwd?: string;
  env: Record<string, string | undefined>;
  signal: AbortSignal;
}
```

The `signal` field tells your spawn function when to tear down the process. Pass it as the `signal` option to Node's `spawn()`, or pass it to your VM or container teardown handler.

This signal does not fire the instant `Options.abortController` aborts. The SDK first closes the process's stdin and waits about two seconds so the CLI can shut down cleanly, then aborts this signal. To react the moment the caller aborts instead, listen on your own `Options.abortController.signal`, which your spawn function can reference from its enclosing scope.

McpSetServersResult

Result of a `setMcpServers()` operation.

```
type McpSetServersResult = {
  added: string[];
  removed: string[];
  errors: Record<string, string>;
};
```

RewindFilesResult

Result of a `rewindFiles()` operation.

```
type RewindFilesResult = {
  canRewind: boolean;
  error?: string;
  filesChanged?: string[];
  insertions?: number;
  deletions?: number;
};
```

SDKStatusMessage

Status update message (e.g., compacting).

```
type SDKStatusMessage = {
  type: "system";
  subtype: "status";
  status: "compacting" | null;
  permissionMode?: PermissionMode;
  uuid: UUID;
  session_id: string;
};
```

`SDKTaskNotificationMessage`

Notification when a background task completes, fails, or is stopped. Background tasks include `run_in_background` Bash commands, **Monitor** watches, and background subagents.

```
type SDKTaskNotificationMessage = {
  type: "system";
  subtype: "task_notification";
  task_id: string;
  tool_use_id?: string;
  status: "completed" | "failed" | "stopped";
  output_file: string;
  summary: string;
  usage?: {
    total_tokens: number;
    tool_uses: number;
    duration_ms: number;
  };
  uuid: UUID;
  session_id: string;
};
```

`SDKToolUseSummaryMessage`

Summary of tool usage in a conversation.

```ts
type SDKToolUseSummaryMessage = {
  type: "tool_use_summary";
  summary: string;
  preceding_tool_use_ids: string[];
  uuid: UUID;
  session_id: string;
};
```

`SDKHookStartedMessage`

Emitted when a hook begins executing.

```ts
type SDKHookStartedMessage = {
  type: "system";
  subtype: "hook_started";
  hook_id: string;
  hook_name: string;
  hook_event: string;
  uuid: UUID;
  session_id: string;
};
```

`SDKHookProgressMessage`

Emitted while a hook is running, with stdout/stderr output.

```ts
type SDKHookProgressMessage = {
  type: "system";
  subtype: "hook_progress";
  hook_id: string;
  hook_name: string;
  hook_event: string;
  stdout: string;
  stderr: string;
  output: string;
  uuid: UUID;
  session_id: string;
};
```

SDKHookResponseMessage

Emitted when a hook finishes executing.

```
type SDKHookResponseMessage = {
  type: "system";
  subtype: "hook_response";
  hook_id: string;
  hook_name: string;
  hook_event: string;
  output: string;
  stdout: string;
  stderr: string;
  exit_code?: number;
  outcome: "success" | "error" | "cancelled";
  uuid: UUID;
  session_id: string;
};
```

SDKToolProgressMessage

Emitted periodically while a tool is executing to indicate progress.

```
type SDKToolProgressMessage = {
  type: "tool_progress";
  tool_use_id: string;
  tool_name: string;
  parent_tool_use_id: string | null;
  elapsed_time_seconds: number;
  task_id?: string;
  uuid: UUID;
  session_id: string;
};
```

SDKAuthStatusMessage

Emitted during authentication flows.

```
type SDKAuthStatusMessage = {
  type: "auth_status";
  isAuthenticating: boolean;
  output: string[];
  error?: string;
  uuid: UUID;
  session_id: string;
};
```

`SDKTaskStartedMessage`

Emitted when a background task begins. The `task_type` field is `"local_bash"` for background Bash commands and **Monitor** watches, `"local_agent"` for subagents, or `"remote_agent"`.

```
type SDKTaskStartedMessage = {
  type: "system";
  subtype: "task_started";
  task_id: string;
  tool_use_id?: string;
  description: string;
  task_type?: string;
  uuid: UUID;
  session_id: string;
};
```

`SDKTaskProgressMessage`

Emitted periodically while a subagent or background task is running. The `summary` field is populated only when `agentProgressSummaries` is enabled.

```ts
type SDKTaskProgressMessage = {
  type: "system";
  subtype: "task_progress";
  task_id: string;
  tool_use_id?: string;
  description: string;
  subagent_type?: string;
  usage: {
    total_tokens: number;
    tool_uses: number;
    duration_ms: number;
  };
  last_tool_name?: string;
  summary?: string;
  uuid: UUID;
  session_id: string;
};
```

`SDKTaskUpdatedMessage`

Emitted when a background task's state changes, such as when it transitions from `running` to `completed`. Merge `patch` into your local task map keyed by `task_id`. The `end_time` field is a Unix epoch timestamp in milliseconds, comparable with `Date.now()`.

```ts
type SDKTaskUpdatedMessage = {
  type: "system";
  subtype: "task_updated";
  task_id: string;
  patch: {
    status?: "pending" | "running" | "completed" | "failed" |
"killed";
    description?: string;
    end_time?: number;
    total_paused_ms?: number;
    error?: string;
    is_backgrounded?: boolean;
  };
  uuid: UUID;
  session_id: string;
};
```

`SDKFilesPersistedEvent`

Emitted when file checkpoints are persisted to disk.

```
type SDKFilesPersistedEvent = {
  type: "system";
  subtype: "files_persisted";
  files: { filename: string; file_id: string }[];
  failed: { filename: string; error: string }[];
  processed_at: string;
  uuid: UUID;
  session_id: string;
};
```

`SDKRateLimitEvent`

Emitted when the session encounters a rate limit.

```
type SDKRateLimitEvent = {
  type: "rate_limit_event";
  rate_limit_info: {
    status: "allowed" | "allowed_warning" | "rejected";
    resetsAt?: number;
    utilization?: number;
    errorCode?: "credits_required";
    canUserPurchaseCredits?: boolean;
    hasChargeableSavedPaymentMethod?: boolean;
  };
  uuid: UUID;
  session_id: string;
};
```

When `errorCode` is `"credits_required"`, the rejection is from a claude.ai subscription whose included usage is exhausted, and the session cannot continue until the user buys usage credits. `canUserPurchaseCredits` indicates whether the authenticated user can buy credits for the account, and `hasChargeableSavedPaymentMethod` indicates whether a saved payment method is on file. All three fields are absent on rate-limit events that are not credits-required rejections. Requires Claude Code v2.1.181 or later.

`SDKLocalCommandOutputMessage`

Output from a local slash command (for example, `/voice` or `/usage`). Displayed as assistant-style text in the transcript.

```
type SDKLocalCommandOutputMessage = {
  type: "system";
  subtype: "local_command_output";
  content: string;
  uuid: UUID;
  session_id: string;
};
```

`SDKCommandsChangedMessage`

Emitted when the set of available commands changes mid-session, such as when skills are discovered as the agent enters a subdirectory. The `commands` array is the full updated list, so replace any cached command list with this payload. Calling `supportedCommands()` again is not equivalent: that method returns the snapshot captured at initialization and does not reflect mid-session changes.

```
type SDKCommandsChangedMessage = {
  type: "system";
  subtype: "commands_changed";
  commands: SlashCommand[];
  uuid: UUID;
  session_id: string;
};
```

`SDKPromptSuggestionMessage`

Emitted after each turn when `promptSuggestions` is enabled. Contains a predicted next user prompt.

```ts
type SDKPromptSuggestionMessage = {
  type: "prompt_suggestion";
  suggestion: string;
  uuid: UUID;
  session_id: string;
};
```

`AbortError`

Custom error class for abort operations.

```ts
class AbortError extends Error {}
```

Sandbox Configuration

`SandboxSettings`

Configuration for sandbox behavior. Use this to enable command sandboxing and configure network restrictions programmatically.

```ts
type SandboxSettings = {
  enabled?: boolean;
  failIfUnavailable?: boolean;
  autoAllowBashIfSandboxed?: boolean;
  excludedCommands?: string[];
  allowUnsandboxedCommands?: boolean;
  network?: SandboxNetworkConfig;
  filesystem?: SandboxFilesystemConfig;
  ignoreViolations?: Record<string, string[]>;
  enableWeakerNestedSandbox?: boolean;
  ripgrep?: { command: string; args?: string[] };
};
```

Property	Type	Default	Description
`enabled`	`boolean`	`false`	Enable sandbox mode for command execution
`failIfUnavailable`	`boolean`	`true`	Stop at startup if `enabled` is `true` but the sandbox can't start. Set `false` to

Property	Type	Default	Description
			fall back to unsandboxed execution with a warning on stderr
`autoAllowBashIfSandboxed`	`boolean`	`true`	Auto-approve bash commands when sandbox is enabled
`excludedCommands`	`string[]`	`[]`	Commands that always bypass sandbox restrictions (e.g., `['docker']`). These run unsandboxed automatically without model involvement
`allowUnsandboxedCommands`	`boolean`	`true`	Allow the model to request running commands outside the sandbox. When `true`, the model can set `dangerouslyDisableSandbox` in tool input, which falls back to the **permissions system**
`network`	`SandboxNetworkConfig`	`undefined`	Network-specific sandbox configuration
`filesystem`	`SandboxFilesystemConfig`	`undefined`	Filesystem-specific sandbox configuration for read/write restrictions
`ignoreViolations`	`Record<string, string[]>`	`undefined`	Map of violation categories to patterns to ignore (e.g., `{ file: ['/tmp/*'], network: ['localhost'] }`)
`enableWeakerNestedSandbox`	`boolean`	`false`	Enable a weaker nested sandbox for compatibility
`ripgrep`	`{ command: string; args?: string[] }`	`undefined`	Custom ripgrep binary configuration for sandbox environments

The sandbox depends on platform support and, on Linux, tools like `bubblewrap` and `socat`. When `enabled` is `true` and the sandbox can't start, `query()` reports a `result` message with `subtype: "error_during_execution"` and the reason in `errors`, then stops. Watch for that subtype rather than expecting `query()` to throw before yielding messages.

To run unsandboxed instead, set `failIfUnavailable: false`.

Example usage

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Build and test my project",
  options: {
    sandbox: {
      enabled: true,
      autoAllowBashIfSandboxed: true,
      network: {
        allowLocalBinding: true
      }
    }
  }
})) {
  if ("result" in message) console.log(message.result);
}
```

> ⚠ **Unix socket security:** The `allowUnixSockets` option can grant access to powerful system services. For example, allowing `/var/run/docker.sock` effectively grants full host system access through the Docker API, bypassing sandbox isolation. Only allow Unix sockets that are strictly necessary and understand the security implications of each.

`SandboxNetworkConfig`

Network-specific configuration for sandbox mode. These settings apply to sandboxed Bash commands when `enabled` is `true` in the parent `SandboxSettings`. They do not restrict the WebFetch tool, which uses **permission rules** instead.

```
type SandboxNetworkConfig = {
  allowedDomains?: string[];
  deniedDomains?: string[];
  allowManagedDomainsOnly?: boolean;
  allowLocalBinding?: boolean;
  allowUnixSockets?: string[];
  allowAllUnixSockets?: boolean;
  httpProxyPort?: number;
  socksProxyPort?: number;
};
```

Property	Type	Default	Description
`allowedDomains`	`string[]`	`[]`	Domain names that sandboxed processes can access
`deniedDomains`	`string[]`	`[]`	Domain names that sandboxed processes cannot access. Takes precedence over `allowedDomains`
`allowManagedDomainsOnly`	`boolean`	`false`	Managed-settings only. When set in **managed settings**, only `allowedDomains` entries from managed settings are honored and entries from user, project, or local settings are ignored. Has no effect when set via SDK options
`allowLocalBinding`	`boolean`	`false`	Allow processes to bind to local ports (e.g., for dev servers)
`allowUnixSockets`	`string[]`	`[]`	Unix socket paths that processes can access (e.g., Docker socket)
`allowAllUnixSockets`	`boolean`	`false`	Allow access to all Unix sockets
`httpProxyPort`	`number`	`undefined`	HTTP proxy port for network requests
`socksProxyPort`	`number`	`undefined`	SOCKS proxy port for network requests

> The built-in sandbox proxy enforces `allowedDomains` based on the requested hostname and does not terminate or inspect TLS traffic, so techniques such as **domain fronting** can potentially bypass it. See **Sandboxing security limitations** for details and **Secure deployment** for configuring a TLS-terminating proxy.

`SandboxFilesystemConfig`

Filesystem-specific configuration for sandbox mode.

```
type SandboxFilesystemConfig = {
  allowWrite?: string[];
  denyWrite?: string[];
  denyRead?: string[];
};
```

Property	Type	Default	Description
`allowWrite`	`string[]`	`[]`	File path patterns to allow write access to
`denyWrite`	`string[]`	`[]`	File path patterns to deny write access to
`denyRead`	`string[]`	`[]`	File path patterns to deny read access to

Permissions Fallback for Unsandboxed Commands

When `allowUnsandboxedCommands` is enabled, the model can request to run commands outside the sandbox by setting `dangerouslyDisableSandbox: true` in the tool input. These requests fall back to the existing permissions system, meaning your `canUseTool` handler is invoked, allowing you to implement custom authorization logic.

`excludedCommands` **vs** `allowUnsandboxedCommands`**:**

- `excludedCommands`: A static list of commands that always bypass the sandbox automatically (e.g., `['docker']`). The model has no control over this.
- `allowUnsandboxedCommands`: Lets the model decide at runtime whether to request unsandboxed execution by setting `dangerouslyDisableSandbox: true` in the tool input.

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Deploy my application",
  options: {
    sandbox: {
      enabled: true,
      allowUnsandboxedCommands: true // Model can request
unsandboxed execution
    },
    permissionMode: "default",
    canUseTool: async (tool, input) => {
      // Check if the model is requesting to bypass the sandbox
      if (tool === "Bash" && input.dangerouslyDisableSandbox) {
        // The model is requesting to run this command outside
the sandbox
        console.log(`Unsandboxed command requested:
${input.command}`);

        if (isCommandAuthorized(input.command)) {
          return { behavior: "allow" as const, updatedInput:
input };
        }
        return {
          behavior: "deny" as const,
          message: "Command not authorized for unsandboxed
execution"
        };
      }
      return { behavior: "allow" as const, updatedInput: input };
    }
  }
})) {
  if ("result" in message) console.log(message.result);
}
```

This pattern enables you to:

- **Audit model requests:** Log when the model requests unsandboxed execution
- **Implement allowlists:** Only permit specific commands to run unsandboxed
- **Add approval workflows:** Require explicit authorization for privileged operations

⚠ Commands running with `dangerouslyDisableSandbox: true` have full system access. Ensure your `canUseTool` handler validates these requests carefully.

If `permissionMode` is set to `bypassPermissions` and `allowUnsandboxedCommands` is enabled, the model can autonomously execute commands outside the sandbox without approval prompts (an explicit `ask` **rule** still forces one). This combination effectively allows the model to escape sandbox isolation silently.

See also

- **SDK overview** - General SDK concepts
- **Python SDK reference** - Python SDK documentation
- **CLI reference** - Command-line interface
- **Common workflows** - Step-by-step guides